



المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Ouvrage pédagogique

Développement web avec Django

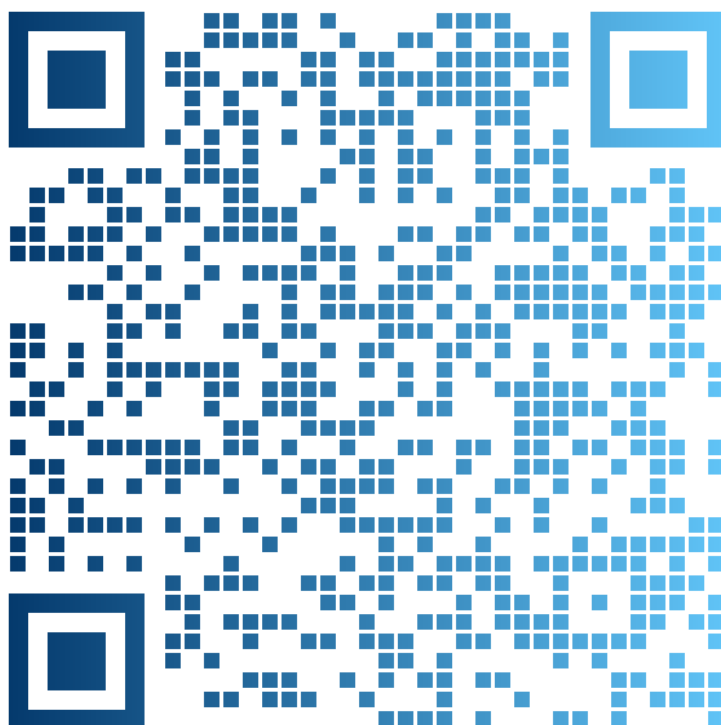
Conception et développement d'applications web

M'barek IAOUSSE

École Supérieure de Technologie de Casablanca
Université Hassan II de Casablanca

Édition 2026

Version en ligne



Scannez ce code pour accéder à la version en ligne.

Cet ouvrage pédagogique a été élaboré comme support du cours *Développement web avec Django*, proposé dans le cadre de la **Formation continue ASERCS** à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur : M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de cet ouvrage électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](#).

Autorisations accordées :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Sous réserve de citer l'auteur.

© M'barek IAOUSSE (2026)

Pour citer cet ouvrage : M'barek IAOUSSE. (2026). **Développement web avec Django, Conception et développement d'applications web**. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Le présent ouvrage est initialement issu des supports de cours **Développement web avec Django**, proposés dans le cadre de la **Formation continue ASERCS** à l'École Supérieure de Technologie de Casablanca (ESTC). Cependant, il est conçu pour servir l'autoformation et guider l'enseignement.

Ce cours s'adresse aux personnes qui possèdent des bases en programmation Python et souhaitent apprendre à construire des applications web. Une connaissance préalable de Django n'est pas nécessaire.

Le parcours accompagne pas à pas la création de **BiblioCours**, une application de gestion et de consultation de cours, depuis la mise en place du projet jusqu'à sa préparation pour la production.

L'ouvrage présente l'architecture d'une application Django, le routage des requêtes, les modèles et l'ORM, les migrations, les vues et templates, les formulaires et l'administration, puis l'authentification, les pratiques de sécurité, les tests et le déploiement. L'objectif est de développer une démarche rigoureuse pour concevoir des applications web maintenables et fiables.

Prérequis

Le contenu de cet ouvrage est organisé pour des personnes ayant déjà une base en :

- la programmation en Python (variables, conditions, boucles, fonctions et structures de données de base) ;
- le HTML et les notions élémentaires du fonctionnement du Web.

Une expérience préalable avec Django n'est pas requise : les notions nécessaires sont introduites progressivement et mises en pratique tout au long du cours.

Ressources complémentaires

La documentation officielle de [Django](#) constitue la ressource de référence pour approfondir les fonctionnalités du framework et consulter les informations correspondant à chaque version.

Les exemples et exercices pratiques de cet ouvrage sont développés en Python avec Django. Ils suivent la réalisation d'une même application afin de relier les concepts aux étapes concrètes du développement web.

Note

Les versions de Python et de Django évoluent. Suivez les versions et les consignes d'installation précisées dans le chapitre de démarrage, et consultez la documentation officielle en cas de différence avec votre environnement.

Sommaire

Préface	i
Démarrer proprement avec Django et uv	1
Ce que nous allons construire	1
Pourquoi Django ?	2
Pourquoi uv remplace venv et pip dans ce cours	3
Créer le projet Django	6
Créer l'application cours	7
Versionner ce qui doit l'être	8
Checkpoint	9
Erreurs fréquentes	9
À retenir	9
Exercices	10
Comprendre l'architecture Django et le routage	11
Où en sommes-nous ?	11
Le chemin complet d'une requête	11
Quelques notions HTTP indispensables	12
Les apps : découper par responsabilité	13
Lire settings.py plutôt que le copier	13
Déclarer les premières routes	15
Écrire une vue minimale	16
Comprendre la résolution des URLs	17
Fabriquer une URL : reverse et les noms de routes	17
Un premier test automatique	18
Requêtes et réponses : les distinctions importantes	19
Premier modèle mental à retenir	19
Checkpoint	20
Erreurs fréquentes	20
À retenir	20
Exercices	20
Modéliser les données avec l'ORM et les migrations	22
Où en sommes-nous ?	22
Penser d'abord au domaine	22
Écrire les modèles de BiblioCours	23
Choisir on_delete	25
Valider au niveau du modèle	25
Les migrations : l'historique du schéma	26
Créer et lire des objets dans le shell	27
Modifier les données avec prudence	30
Concevoir avant de migrer	31
Checkpoint	31
Erreurs fréquentes	31
À retenir	31
Exercices	32
Construire les pages : vues, URLs et templates	33
Où en sommes-nous ?	33
De la donnée à une page HTML	33
Le langage de templates en bref	33

Les vues de liste et de détail	34
Organiser les templates	36
Le template parent	37
Afficher une collection avec <code>liste.html</code>	38
Afficher le détail	39
Rechercher avec un paramètre GET	40
Lire un bug de template	40
Checkpoint	41
Erreurs fréquentes	41
À retenir	41
Exercices	42
Saisir et administrer les données : formulaires et admin	43
Où en sommes-nous ?	43
Pourquoi un formulaire HTML ne suffit pas	43
Créer un <code>ModelForm</code>	44
La vue de création	45
Le template du formulaire	46
Modifier un objet existant	47
Supprimer : préférer une confirmation explicite	48
L'administration Django	49
Messages utilisateur	50
Checkpoint	51
Erreurs fréquentes	51
À retenir	52
Exercices	52
Authentification, sécurité et fichiers statiques	53
Où en sommes-nous ?	53
Authentifier n'est pas autoriser	53
Ajouter les routes d'authentification	54
Protéger une vue et une action	55
CSRF : protéger les requêtes qui changent l'état	57
Échappement, injection et entrées utilisateur	57
Secrets et configuration	59
Fichiers statiques	61
Téléversements et fichiers média	61
Checklist de sécurité locale	62
Checkpoint	62
Erreurs fréquentes	62
À retenir	63
Exercices	63
Tester, diagnostiquer et faire évoluer le code	65
Un test décrit un contrat	65
Lancer la suite	65
Tester le modèle	65
Tester les URLs et les vues	66
Tester l'authentification et les permissions	67
Tester un formulaire	68
Déboguer sans deviner	68
Qualité automatique avec <code>ruff</code>	69
Une petite stratégie de test	69
Exercices	69
Préparer la production et continuer l'apprentissage	70
Développement et production n'ont pas le même but	70
Séparer la configuration du code	70
Exécuter les vérifications Django	71

Base de données et migrations	71
Fichiers statiques	72
WSGI, ASGI et processus	72
Journalisation et erreurs	72
Déployer avec un <code>pyproject.toml</code>	73
Une checklist finale	73
Aller plus loin : une progression raisonnable	73
Projet de synthèse	74
Exercices	74
References	75

Démarrer proprement avec Django et uv

i Objectifs d'apprentissage

À la fin de ce chapitre, vous saurez :

- décrire ce qu'est une requête HTTP et ce que Django fait entre la requête et la réponse ;
- expliquer pourquoi un projet Python doit avoir un environnement isolé et reproductible ;
- créer un projet avec uv, installer Django et lire les fichiers `pyproject.toml` et `uv.lock` ;
- distinguer un **projet** Django d'une **application** Django ;
- démarrer le serveur de développement et vérifier la configuration avec `check` ;
- décider quels fichiers doivent (ou non) être versionnés avec Git.

Ce que nous allons construire

Ce cours suit la construction progressive de **BiblioCours**, une petite bibliothèque de cours en ligne. L'application permettra finalement de :

- classer des cours par matière ;
- afficher une liste puis le détail d'un cours ;
- gérer les données depuis l'administration Django ;
- permettre à des utilisateurs authentifiés de contribuer ;
- tester les comportements importants avant de les faire évoluer.

Le projet est volontairement assez petit pour rester lisible, mais assez réaliste pour faire apparaître les problèmes qui comptent dans une application web : organisation du code, données relationnelles, validation, sécurité, tests et déploiement.

La carte du cours

Chaque chapitre ajoute une capacité à BiblioCours et s'appuie sur le précédent. Le tableau ci-dessous sert de repère : revenez-y quand vous vous demandez « pourquoi fait-on cela maintenant ? ».

Chapitre	Question posée	Ce que BiblioCours gagne
1. Démarrer	Comment partir sur une base saine et reproductible ?	un projet, une app, un dépôt Git
2. Architecture et routage	Comment une URL atteint-elle du code ?	des routes nommées et des premières vues
3. Modèles et migrations	Comment décrire et stocker les données ?	les modèles Matiere et Cours
4. Vues et templates	Comment afficher les données ?	liste, détail, recherche, pagination
5. Formulaires et admin	Comment saisir et gérer les données ?	création, modification, suppression, admin
6. Sécurité	Qui a le droit de faire quoi ?	connexion, permissions, secrets, fichiers
7. Tests	Comment s'assurer que ça marche encore ?	une suite de tests et un contrôle qualité

Chapitre	Question posée	Ce que BiblioCours gagne
8. Production	Comment le mettre en ligne sans danger ?	une configuration et une checklist de déploiement

Prérequis et version du cours

On suppose que vous savez lire du HTML et écrire des programmes Python simples (fonctions, classes, imports, exceptions et modules). Les exemples visent Django 5.2 et Python 3.12 ou une version plus récente compatible. Les idées restent valables pour les versions voisines, mais les détails de configuration doivent toujours être vérifiés dans la documentation de la version installée ([Django Software Foundation 2025b](#)).

Comment utiliser ce cours

Chaque chapitre suit la même structure : des **objectifs**, un rappel de l'**état de départ** du projet, des explications avec du code à reproduire, un **checkpoint** pour vérifier que tout fonctionne, un tableau d'**erreurs fréquentes**, un résumé « **à retenir** » et des **exercices**.

Quelques conventions :

- les commandes fonctionnent dans un terminal macOS/Linux ou PowerShell, sauf indication contraire. Sous PowerShell, utilisez `Get-ChildItem -Force` à la place de `ls -a`, `Get-Content fichier -TotalCount 30` à la place de `head -30 fichier`, et `Remove-Item main.py` à la place de `rm main.py` ;
- toutes les commandes se lancent **à la racine du projet**, là où se trouve `manage.py` ;
- recopiez le code plutôt que de le copier-coller : vous remarquerez davantage de choses en le tapant.

Pour travailler en autonomie, terminez le **checkpoint** de chaque chapitre avant de passer au suivant. Si un résultat diffère, relisez la première erreur complète dans le terminal, vérifiez que vous êtes à la racine du projet et relancez la dernière commande après correction. Ne poursuivez pas en empilant des changements sur une étape qui ne fonctionne pas encore.

Pourquoi Django ?

Une application web reçoit des requêtes HTTP, décide quoi faire, lit ou modifie des données, puis renvoie une réponse HTTP. On pourrait programmer chacune de ces opérations à la main. Django fournit plutôt une structure cohérente :

1. un routeur associe une URL à du code Python ;
2. une vue coordonne le traitement de la requête ;
3. un modèle décrit les données et dialogue avec la base ;
4. un moteur de templates produit le HTML ;
5. des composants prêts à l'emploi prennent en charge l'administration, l'authentification, les sessions et une partie des protections de sécurité.

On dit souvent que Django est un framework « avec les piles incluses » : là où d'autres outils vous demandent de choisir et d'assembler une bibliothèque pour chaque besoin, Django propose une réponse intégrée et cohérente.

Cette approche réduit le code répétitif, mais elle impose aussi des conventions. Apprendre Django revient donc moins à mémoriser des fonctions qu'à comprendre le rôle de chaque couche et le moment où elle doit intervenir. Le tutoriel officiel suit la même logique en construisant une application par étapes ([Django Software Foundation 2025j](#)).

Une requête, concrètement

HTTP est un dialogue en texte entre un client (le navigateur) et un serveur. Lorsque l'on ouvre `http://127.0.0.1:8000/cours/`, le navigateur envoie approximativement :

```
GET /cours/ HTTP/1.1
Host: 127.0.0.1:8000
Accept: text/html
```

Cette requête comporte trois parties : une **ligne de requête** (la méthode `GET`, le chemin `/cours/` et la version du protocole), des **en-têtes** (`Host`, `Accept`, ...) qui décrivent le contexte, et parfois un **corps** (absent ici, mais présent quand on envoie un formulaire).

Django reçoit cette requête, cherche une route correspondante, appelle la vue, éventuellement interroge la base de données, rend un template et renvoie par exemple :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!doctype html>
<html lang="fr">...</html>
```

La réponse comporte aussi trois parties : une **ligne de statut** (`200 OK` signifie « tout s'est bien passé »), des **en-têtes** et un **corps** (le HTML).

Le code du cours rendra ce chemin observable. À chaque étape, nous chercherons à répondre à trois questions : **quelle est l'entrée ? quelle décision est prise ? quelle sortie est produite ?**

Pourquoi uv remplace venv et pip dans ce cours

Le problème : les dépendances en conflit

Imaginez que vous travaillez sur deux projets. Le premier exige Django 4.2, le second Django 5.2. Si les deux utilisent le même Python « global » de votre machine, l'installation de l'un casse l'autre. Imaginez maintenant qu'un collègue clone votre projet : il installe Django aujourd'hui, avec une version légèrement différente de la vôtre, et un comportement change.

Il faut donc :

1. **isoler** les dépendances de chaque projet dans un environnement dédié (un « environnement virtuel ») ;
2. **déclarer** ce dont le projet a besoin ;
3. **enregistrer** les versions exactes installées pour pouvoir les retrouver.

La solution : uv

uv est un gestionnaire rapide de versions Python, d'environnements et de dépendances. Il utilise deux fichiers complémentaires ([Astral 2025](#)) :

Fichier	Rôle	Analogie	Qui l'écrit ?
<code>pyproject.toml</code>	déclare ce que le projet demande	la liste de courses	vous (via <code>uv add</code>)

Fichier	Rôle	Analogie	Qui l'écrit ?
uv.lock	enregistre ce qui a été réellement installé , versions exactes comprises	le ticket de caisse	uv, automatiquement

Nous n'avons donc pas besoin de créer manuellement un dossier `env/`, ni de penser à activer un environnement à chaque terminal : `uv` crée un dossier `.venv/` et l'utilise pour vous.

Installation et vérification

Installez `uv` en suivant la méthode officielle correspondant à votre système. Puis vérifiez que la commande est disponible :

```
uv --version
python --version
```

Sur Windows, PowerShell et Windows Terminal conviennent. Sur macOS et Linux, un terminal courant suffit. Si la commande `uv` n'est pas trouvée après l'installation, redémarrez le terminal afin que le `PATH` (la liste des dossiers où le système cherche les commandes) soit rechargé.

Initialiser le projet

Créez un dossier vide, puis initialisez un projet Python :

```
mkdir bibliocours
cd bibliocours

uv init --python 3.12
```

Sous macOS/Linux, affichez les fichiers, y compris les fichiers cachés, avec `ls -a`. Sous PowerShell, utilisez `Get-ChildItem -Force`. Selon votre version d'`uv`, l'initialisation crée `pyproject.toml`, `.python-version`, `README.md`, `main.py`, un `.gitignore` minimal et peut même initialiser un dépôt Git. Nous n'avons pas besoin de `main.py` (Django fournira `manage.py`) : supprimez-le avec la commande adaptée à votre terminal :

```
# macOS / Linux
rm main.py
```

```
# Windows PowerShell
Remove-Item main.py
```

Le choix de la version de Python doit être explicite : il évite qu'un collègue utilise par hasard une version très différente. Le fichier `.python-version` documente la version attendue sans mettre Python lui-même dans Git. Si cette version n'existe pas encore sur votre machine, `uv` peut l'installer :

```
uv python install 3.12
uv python pin 3.12
```

Ajouter Django

Ajoutez Django comme dépendance de production, c'est-à-dire une dépendance nécessaire pour que l'application fonctionne :

```
uv add "Django>=5.2,<6"
```

La contrainte `>=5.2,<6` signifie « toute version 5.2 ou supérieure, mais pas la 6 ». Elle autorise les corrections de sécurité tout en évitant un changement majeur imprévu. Examinez ensuite les fichiers produits :

```
ls
cat pyproject.toml
head -30 uv.lock
```

Dans `pyproject.toml`, Django apparaît dans la liste des dépendances :

```
[project]
name = "bibliocours"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "django>=5.2,<6",
]
```

Dans `uv.lock`, uv conserve les versions concrètement retenues, y compris les dépendances indirectes (celles dont Django lui-même a besoin, comme `asgiref` et `sqlparse`). Il faut versionner `pyproject.toml` et `uv.lock`; il ne faut pas versionner `.venv/`, qui peut être recréé à tout moment.

Les commandes uv à connaître dès maintenant :

Commande	Effet
<code>uv add paquet</code>	ajoute une dépendance et met à jour <code>pyproject.toml</code> et <code>uv.lock</code>
<code>uv add --dev paquet</code>	ajoute une dépendance de développement (outils de test, de style...)
<code>uv remove paquet</code>	retire une dépendance
<code>uv sync</code>	crée ou met à jour <code>.venv/</code> pour correspondre aux fichiers
<code>uv run commande</code>	exécute une commande dans l'environnement du projet
<code>uv tree</code>	affiche l'arbre des dépendances, directes et indirectes

! `uv run` est le nouveau réflexe

Lancez une commande Python du projet avec `uv run` :

```
uv run python -c "import django; print(django.get_version())"
uv run django-admin --version
```

`uv run` vérifie l'environnement et exécute la commande avec les dépendances du projet. Vous n'avez donc pas à taper `source .venv/bin/activate`. Une activation manuelle reste possible, mais elle n'est plus nécessaire pour suivre ce cours.

Si vous tapez `python manage.py ...` sans `uv run`, vous utilisez peut-être le Python global, où Django

n'est pas installé : l'erreur `ModuleNotFoundError: No module named 'django'` est le signe typique de cet oubli.

Créer le projet Django

Projet et application : deux notions à ne pas confondre

Dans Django, un **projet** représente la configuration globale d'un site. Une **application** est une unité fonctionnelle qui regroupe des modèles, vues, templates et tests. Un même projet peut réunir plusieurs applications : `cours`, `comptes`, `notifications`, etc.

Pour retenir la différence, pensez à une maison. Le projet est le bâtiment avec son électricité, sa plomberie et son adresse : il fournit l'infrastructure commune. Les applications sont les pièces : chacune a une fonction précise. Le projet assemble ; les applications portent les fonctionnalités.

Générer la configuration globale

Créons la configuration globale dans le dossier courant :

```
uv run django-admin startproject config .
```

Le point final signifie « utiliser le dossier courant » plutôt que créer un dossier imbriqué `config/config`.

Nous nommons le paquet de configuration `config` plutôt que `bibliocours` pour éviter la confusion entre le dossier racine et le paquet qui contient `settings.py`. La structure obtenue est :

```
bibliocours/
  .python-version
  pyproject.toml
  uv.lock
  manage.py
  config/
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
```

Le rôle de chaque fichier

- `manage.py` prépare l'accès aux réglages (il indique à Django que ceux-ci se trouvent dans `config.settings`) et expose les commandes Django ;
- `config/settings.py` contient la configuration globale ;
- `config/urls.py` est le routeur principal ;
- `config/asgi.py` et `config/wsgi.py` sont les points d'entrée utilisés par les serveurs applicatifs en production (voir Section) ;
- `__init__.py` indique que `config` est un paquet Python.

Démarrer le serveur de développement

```
uv run python manage.py runserver
```

Ouvrez `http://127.0.0.1:8000/`. Vous devez voir la page d'accueil de Django (une fusée). Dans le terminal, chaque requête reçue est journalisée :

```
[07/Oct/2026 10:15:02] "GET / HTTP/1.1" 200 10731
```

Lisez cette ligne : la méthode, le chemin, la version HTTP, le **statut** de la réponse et sa taille en octets. Ce journal sera votre meilleur ami pour diagnostiquer les problèmes.

i « You have 18 unapplied migration(s) »

Au démarrage, Django affiche un avertissement rouge à propos de migrations non appliquées. C'est normal : les composants fournis par Django (administration, authentification, sessions) ont besoin de tables dans la base de données, qui n'existent pas encore. Nous créerons ces tables au Section , quand nous étudierons les migrations. Ignorez l'avertissement pour l'instant.

Le serveur recharge souvent le code quand un fichier Python est modifié. Il est pratique pour apprendre, mais il ne doit pas être utilisé comme serveur public : il n'est ni dimensionné ni configuré pour la production.

Arrêtez-le avec `Ctrl+C`. Une seconde commande dans un autre terminal peut inspecter le projet :

```
uv run python manage.py check
```

`check` analyse la configuration sans modifier la base de données. Le message attendu est `System check identified no issues (0 silenced)`. Prenez l'habitude de l'exécuter après une modification structurelle.

Créer l'application cours

```
uv run python manage.py startapp cours
```

La nouvelle structure ressemble à ceci :

```
cours/
  __init__.py
  admin.py
  apps.py
  migrations/
    __init__.py
  models.py
  tests.py
  views.py
```

Chaque fichier a une place prévue : `models.py` accueillera les données, `views.py` le traitement des requêtes, `admin.py` la configuration de l'administration, `migrations/` l'historique du schéma de la base. Nous remplirons ces fichiers un à un dans les chapitres suivants.

Une app n'est pas active simplement parce que son dossier existe. Ajoutez-la dans `config/settings.py` :

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "cours",  
]
```

INSTALLED_APPS est la liste des composants que Django doit charger. Tant que `cours` n'y figure pas, Django ignorera ses modèles, ses templates et ses migrations. Dans un projet plus grand, on préfère souvent `"cours.apps.CoursConfig"` : cette forme rend explicite la classe de configuration définie dans `apps.py`. La forme courte fonctionne ici car Django sait retrouver `CoursConfig`.

Relancez `uv run python manage.py check` : si vous aviez fait une faute de frappe dans le nom de l'app, Django la signalerait immédiatement.

Versionner ce qui doit l'être

Si `uv init` n'a pas déjà créé de dépôt Git (cherchez un dossier `.git` avec `ls -a`), initialisez-le :

```
git init
```

Complétez ensuite le fichier `.gitignore` pour qu'il contienne au moins :

```
.venv/  
__pycache__/  
*.py[cod]  
*.sqlite3  
.env  
.DS_Store  
media/  
staticfiles/
```

Voici ce qui doit entrer dans Git, et pourquoi :

Fichier ou dossier	Versionner ?	Raison
code Python, templates, migrations	oui	c'est le projet
<code>pyproject.toml</code> , <code>uv.lock</code>	oui	permettent de recréer l'environnement
<code>.python-version</code>	oui	documente la version de Python attendue
<code>.venv/</code>	non	se recrée avec <code>uv sync</code>
<code>*.sqlite3</code>	non	données de travail locales
<code>media/</code>	non	fichiers téléversés par des utilisateurs
<code>staticfiles/</code>	non	produit par <code>collectstatic</code> , donc régénérable
<code>.env</code>	jamais	contient des secrets

La base SQLite locale et les fichiers téléversés peuvent contenir des données de travail ; ils ne doivent pas être envoyés par défaut. Les secrets ne doivent jamais se trouver dans Git, car l'historique conserve

tout : supprimer un secret dans un commit ultérieur ne l'efface pas du passé. `uv.lock`, en revanche, **doit** être conservé : il sert à reproduire l'installation.

```
git add .
git commit -m "chore: initialiser le projet Django"
git status
```

Un commit est une étape vérifiable, pas une sauvegarde automatique de chaque frappe. Faites un commit lorsque l'application atteint un état cohérent : le projet démarre, le contrôle Django passe et les fichiers nécessaires sont présents. Le préfixe `chore:` suit la convention « Conventional Commits » : `feat:` pour une fonctionnalité, `fix:` pour une correction, `test:` pour des tests, `docs:` pour de la documentation.

Checkpoint

À la fin de ce chapitre, les commandes suivantes doivent fonctionner :

```
uv run python manage.py check
uv run python manage.py runserver
uv run django-admin --version
git status
```

Vérifiez :

- ☐ `check` affiche « no issues » ;
- ☐ la page de la fusée s'affiche sur `http://127.0.0.1:8000/` ;
- ☐ `django-admin --version` affiche une version 5.2.x ;
- ☐ `git status` indique « nothing to commit, working tree clean » ;
- ☐ `.venv/` et `db.sqlite3` n'apparaissent pas dans `git status`.

Erreurs fréquentes

Symptôme	Cause probable	Remède
<code>uv: command not found</code>	terminal ouvert avant l'installation	fermer et rouvrir le terminal
<code>ModuleNotFoundError: No module named 'django'</code>	commande lancée sans <code>uv run</code>	préfixer par <code>uv run</code>
<code>can't open file 'manage.py': Error: That port is already in use</code>	mauvais dossier courant un autre serveur tourne déjà	<code>cd</code> vers la racine du projet l'arrêter, ou <code>runserver 8001</code>
<code>ModuleNotFoundError: No module named 'cours'</code> dossier <code>config/config/</code>	app mal nommée ou absente du dossier oubli du point final de <code>startproject</code>	vérifier le nom dans <code>INSTALLED_APPS</code> recommencer dans un dossier propre

À retenir

- Une requête HTTP a une ligne de requête, des en-têtes et parfois un corps ; la réponse a un statut, des en-têtes et un corps.
- `pyproject.toml` déclare, `uv.lock` enregistre : les deux sont versionnés.
- `uv run` exécute toute commande dans l'environnement du projet.

- Le **projet** assemble et configure ; les **applications** portent les fonctionnalités.
- Une app doit être déclarée dans `INSTALLED_APPS` pour exister.
- **runserver** sert à apprendre, jamais à publier.
- Un secret ne doit jamais entrer dans Git.

Exercices

1. Décrivez avec vos mots la différence entre un projet Django et une app Django. Donnez ensuite trois apps possibles pour un site de e-commerce.
2. Expliquez pourquoi `uv.lock` est versionné alors que `.venv/` ne l'est pas. Que se passerait-il pour un collègue si `uv.lock` était absent ?
3. Pourquoi la commande `startproject config .` se termine-t-elle par un point ? Qu'obtiendrait-on sans lui ?
4. Citez trois raisons pour lesquelles **runserver** ne convient pas à un site public.
5. Dans la ligne de journal "GET /cours/ HTTP/1.1" 404 2345, que signifient les différents éléments ? Que vous dit le 404 ?
6. Supprimez `.venv/` si elle existe, puis relancez une commande avec `uv run`. Que constatez-vous ? Quelle commande aurait-elle pu recréer l'environnement explicitement ?
7. Ajoutez une dépendance de développement avec `uv add --dev ruff`, puis observez la différence entre les dépendances normales et celles de développement dans `pyproject.toml`. Lancez `uv run ruff --version`. Cet outil sera réutilisé au Section .
8. Faites un commit avant et après la création de l'app `cours`, puis utilisez `git show --stat` pour identifier ce que Django a généré.
9. Lancez `uv tree`. Quelles sont les dépendances **indirectes** de Django ? Retrouvez-les dans `uv.lock`.
10. Lancez le serveur sur le port 8001 (`runserver 8001`), puis, dans un second terminal, observez la réponse brute avec `curl -i http://127.0.0.1:8001/` (sous PowerShell, tapez `curl.exe`). Repérez la ligne de statut et les en-têtes.
11. Ouvrez `config/settings.py` et listez cinq réglages dont vous ne comprenez pas encore le rôle. Gardez cette liste : au chapitre suivant, vous pourrez la compléter avec les réponses.
12. Rédigez un fichier `README.md` qui décrit, en cinq commandes maximum, comment un nouveau venu installe et lance BiblioCours. Demandez à une personne de suivre vos instructions sur une machine « propre » et notez où elle bute.

Comprendre l'architecture Django et le routage

Objectifs d'apprentissage

À la fin de ce chapitre, vous saurez :

- décrire le chemin complet d'une requête dans Django et le rôle du modèle, de la vue et du template (MVT) ;
- lire un fichier `settings.py` et expliquer ses sections principales ;
- déclarer des routes avec `path`, `include`, `app_name` et `name` ;
- capturer des paramètres dans une URL grâce aux convertisseurs ;
- expliquer pourquoi l'ordre des routes compte et le démontrer ;
- générer une URL avec `reverse` plutôt que de l'écrire en dur ;
- manipuler `HttpRequest` et `HttpResponse` et diagnostiquer une page qui ne fonctionne pas.

Où en sommes-nous ?

Au chapitre précédent, nous avons obtenu un projet qui démarre, une app `cours` déclarée dans `INSTALLED_APPS` et un dépôt Git propre. L'app est encore vide : aucune page n'existe en dehors de la fusée de Django. Ce chapitre construit les **routes** (qui répondent à « quelle URL mène à quel code ? ») et les premières **vues**, sans base de données ni HTML : nous voulons comprendre le mécanisme avant d'ajouter des couches.

Le chemin complet d'une requête

Une bonne application Django n'est pas une collection de fichiers que l'on modifie au hasard. Elle est une chaîne de responsabilités. Pour une requête vers `/cours/python-pour-debutants/`, le parcours conceptuel est :

```
navigateur
  requête HTTP

config/urls.py
  route choisie

vue Python
  lecture ou écriture

modèle / ORM    base de données

template        contexte Python
  HTML

navigateur
```

Django appelle souvent cette organisation **MVT** (Modèle-Vue-Template) :

Élément Django	Responsabilité	Analogie MVC
Model	données, contraintes et accès à la base	Model
View	traitement de la requête et choix de la réponse	Controller
Template	représentation HTML	View

La comparaison avec MVC est utile mais imparfaite : le mot « vue » désigne dans Django ce que d'autres frameworks appellent un contrôleur. Le nom importe moins que la règle : une vue ne devrait pas contenir tout le HTML, un template ne devrait pas interroger directement la base, et un modèle ne devrait pas connaître la mise en page. Le découpage rend chaque partie remplaçable et testable ([Django Software Foundation 2025c](#)). Les bonnes pratiques de conception Django insistent aussi sur la lisibilité, la simplicité et le respect des conventions du framework ([Greenfeld et Roy Greenfeld 2017](#)).

À quelle couche appartient ce code ?

Face à une règle ou à un morceau de code, demandez-vous où le ranger :

Si cela concerne...	Cela va dans...	Exemple dans BiblioCours
la forme des données et leurs contraintes	le modèle	un slug est unique
la décision à prendre pour une requête	la vue	n'afficher que les cours publiés
la mise en page HTML	le template	la carte d'un cours
quelle URL mène où	le routeur (<code>urls.py</code>)	<code>/cours/<slug>/</code>
la validation d'une saisie	le formulaire (chapitre 5)	titre de 5 caractères minimum

Quelques notions HTTP indispensables

Deux informations accompagnent chaque échange : la **méthode** de la requête et le **code de statut** de la réponse.

Méthode	Usage habituel	Modifie des données ?
GET	lire une page	non
POST	envoyer un formulaire, déclencher une action	oui

Statut	Signification	Exemple
200 OK	succès	la page s'affiche
301 / 302	redirection permanente / temporaire	après un formulaire valide
403 Forbidden	identifié (ou non) mais interdit	permission manquante
404 Not Found	ressource inconnue	slug qui n'existe pas
405 Method Not Allowed	méthode non acceptée	GET sur une vue réservée à POST
500 Internal Server Error	erreur dans le code serveur	exception non gérée

Retenez la famille du premier chiffre : **2xx** succès, **3xx** redirection, **4xx** erreur du client (la requête est incorrecte), **5xx** erreur du serveur (le code est en cause). Cette lecture vous aidera à savoir où chercher.

Les apps : découper par responsabilité

L'app `cours` aura les responsabilités liées au catalogue pédagogique. Dans une application qui grandit, on pourrait ajouter :

<code>comptes/</code>	<code>utilisateurs, profils, permissions</code>
<code>cours/</code>	<code>matières et cours</code>
<code>inscriptions/</code>	<code>suivi de la progression</code>
<code>notifications/</code>	<code>courriels et alertes</code>

Une app bien découpée peut être déplacée dans un autre projet avec peu de modifications. Il ne faut pas créer une app pour chaque modèle : on crée une app pour un domaine fonctionnel cohérent. Ici, `Matiere` et `Cours` forment ensemble un seul domaine, le catalogue : une seule app suffit.

Lire `settings.py` plutôt que le copier

Le fichier `config/settings.py` configure le comportement du projet. Ce n'est pas un fichier à mémoriser, mais à savoir **lire**. Voici ses sections importantes, dans l'ordre :

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = "à-remplacer-en-production"
DEBUG = True
ALLOWED_HOSTS = []

INSTALLED_APPS = [
    # Apps Django fournies
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # App du projet
    "cours",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

ROOT_URLCONF = "config.urls"

TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
```

```
    "OPTIONS": {
        "context_processors": [
            "django.template.context_processors.request",
            "django.contrib.auth.context_processors.auth",
            "django.contrib.messages.context_processors.messages",
        ],
    },
},
],
WSGI_APPLICATION = "config.wsgi.application"
```

Passons-les en revue :

- `BASE_DIR` est le dossier racine du projet. On construit les autres chemins à partir de lui, pour que le projet fonctionne quel que soit son emplacement ;
- `INSTALLED_APPS` active les composants ;
- `MIDDLEWARE` forme une pile traversée par les requêtes et les réponses (voir l'encadré ci-dessous) ;
- `ROOT_URLCONF` indique quel routeur commencer à lire ;
- `TEMPLATES` indique comment trouver et rendre les templates ;
- `WSGI_APPLICATION` désigne le point d'entrée pour les serveurs de production.

💡 Les middlewares : une pelure d'oignon

Chaque middleware est une couche qui enveloppe la vue. La requête traverse les couches **de haut en bas** de la liste avant d'atteindre la vue ; la réponse les retraverse **de bas en haut**.

```
requête → Security → Session → Common → Csrf → Auth → ... → VUE
réponse ← Security ← Session ← Common ← Csrf ← Auth ← ... ← VUE
```

C'est ainsi que `request.user` existe dans toutes les vues (grâce à `SessionMiddleware` et `AuthenticationMiddleware`) et que les formulaires sont protégés contre le CSRF (grâce à `CsrfViewMiddleware`). L'ordre compte : un middleware peut dépendre de ce qu'un autre a préparé avant lui.

Ce qui se trouve plus bas dans le fichier

Trois autres réglages produits par `startproject` méritent un regard :

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}

LANGUAGE_CODE = "en-us"
TIME_ZONE = "UTC"
USE_TZ = True

STATIC_URL = "static/"
```

- `DATABASES` indique **quelle base** utiliser. Par défaut, SQLite, un simple fichier `db.sqlite3` : idéal pour apprendre, car il n'y a rien à installer ;
- `LANGUAGE_CODE` détermine la langue des messages de Django (erreurs de formulaire, administration) ;

- `TIME_ZONE` et `USE_TZ` gèrent les dates. Avec `USE_TZ = True`, Django stocke les dates en UTC dans la base et les convertit dans le fuseau configuré à l’affichage ;
- `STATIC_URL` est le préfixe des fichiers CSS et JavaScript (voir Section).

Pour que BiblioCours parle français, modifiez :

```
LANGUAGE_CODE = "fr"
TIME_ZONE = "Europe/Paris" # remplacez par votre fuseau horaire
```

Redémarrez le serveur et ouvrez `/admin/` : la page de connexion est désormais en français.

Le secret et le mode debug

`SECRET_KEY` est utilisé par plusieurs mécanismes cryptographiques de Django (signature des sessions, des jetons CSRF...). Il ne doit pas être publié. `DEBUG = True` affiche des informations très détaillées en cas d’erreur ; c’est utile localement et dangereux sur Internet. `ALLOWED_HOSTS` liste les noms de domaine que Django accepte de servir ; vide, il n’accepte que `localhost` et `127.0.0.1` tant que `DEBUG` est actif. Nous apprendrons à protéger ces trois réglages au Section , et à les configurer pour la production au Section .

Déclarer les premières routes

Le routeur principal reçoit une liste de motifs. Modifiez `config/urls.py` :

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("cours/", include("cours.urls")),
]
```

Créez ensuite `cours/urls.py` :

```
from django.urls import path

from . import views

app_name = "cours"

urlpatterns = [
    path("", views.accueil, name="accueil"),
]
```

La fonction `path(route, vue, name=...)` associe un motif à une vue. Trois détails sont essentiels :

1. `include` délègue un préfixe à l’app : toute URL commençant par `cours/` est confiée à `cours/urls.py`, qui ne voit alors que la suite du chemin ;
2. `app_name` crée un **espace de noms** : la route s’appellera `cours:accueil`, pas simplement `accueil` ;
3. `name` donne un identifiant stable à la route.

Un template doit utiliser le nom de la route, pas recopier son URL. Ainsi, si `cours/` devient `catalogue/`, les liens générés par `{% url %}` restent cohérents après la modification du routeur.

i Pourquoi tous ces / finaux ?

Django a l'habitude de terminer les URLs par /. Si vous demandez `/cours` (sans le / final), `CommonMiddleware` répond par une redirection vers `/cours/`. Cela évite que deux adresses différentes présentent la même page, ce que les moteurs de recherche n'apprécient pas. Écrivez donc toujours vos routes avec un / final.

Écrire une vue minimale

Dans `cours/views.py` :

```
from django.http import HttpRequest, HttpResponse

def accueil(request: HttpRequest) -> HttpResponse:
    return HttpResponse("Bienvenue dans BiblioCours")
```

Démarrez le serveur et ouvrez `http://127.0.0.1:8000/cours/`. Vous devriez voir le message. Une vue est une simple fonction qui reçoit au minimum un objet `HttpRequest` et renvoie un objet de réponse. `HttpResponse` est idéal pour comprendre le mécanisme, mais une application HTML utilisera plutôt `render`, présenté au Section .

Observez le journal du terminal : une ligne `"GET /cours/ HTTP/1.1" 200 ...` confirme que la requête est arrivée, a été routée et a produit un 200.

Paramètres capturés dans l'URL

Django sait convertir une partie de l'URL avant d'appeler la vue. Ajoutez une route de détail à `cours/urls.py` :

```
urlpatterns = [
    path("", views.accueil, name="accueil"),
    path("<slug:slug>/", views.detail_cours, name="detail"),
]
```

La syntaxe `<convertisseur:nom>` capture un segment de l'URL et le transmet à la vue comme argument nommé. Pour `/cours/python-pour-debutants/`, Django appelle `detail_cours(request, slug="python-pour-debutants")`. Ajoutez la vue :

```
def detail_cours(request: HttpRequest, slug: str) -> HttpResponse:
    return HttpResponse(f"Le cours demandé est : {slug}")
```

Le nom entre chevrons doit correspondre exactement au nom du paramètre de la vue. Les convertisseurs disponibles sont :

Convertisseur	Accepte	Exemple de segment valide	Valeur Python
<code>str</code>	tout texte sans / (défaut)	<code>bonjour</code>	<code>str</code>
<code>int</code>	entiers positifs ou nuls	<code>12</code>	<code>int</code>
<code>slug</code>	lettres, chiffres, - et _	<code>python-pour-debutants</code>	<code>str</code>
<code>uuid</code>	un UUID formaté	<code>075194d3-6885-417e-a8a8-8760931e272f00</code>	<code>str</code>
<code>path</code>	texte avec des /	<code>docs/guide/intro</code>	<code>str</code>

Convertisseur	Accepte	Exemple de segment valide	Valeur Python
---------------	---------	---------------------------	---------------

Un segment qui ne correspond pas au convertisseur n'est pas une erreur de la vue : la route ne correspond simplement pas, et Django essaie la suivante, puis renvoie une 404 s'il n'en reste aucune. Une URL porte donc une information structurée, et non une chaîne que la vue doit parser elle-même.

💡 Question de conception

Le projet utilisera un **slug** (`/cours/python-pour-debutants/`) plutôt qu'un identifiant entier (`/cours/12/`). L'URL est plus lisible, plus parlante pour un moteur de recherche et ne révèle pas le nombre de cours. L'identifiant entier, lui, est plus compact et immuable. Le choix de l'URL est à la fois une décision technique, une décision d'ergonomie et une décision de référencement.

Comprendre la résolution des URLs

Django parcourt `urlpatterns` de haut en bas et s'arrête au **premier** motif correspondant. L'ordre compte. Ajoutons une page statique « À propos » :

```
urlpatterns = [
    path("", views.accueil, name="accueil"),
    path("a-propos/", views.a_propos, name="a_propos"),
    path("<slug:slug>/", views.detail_cours, name="detail"),
]
```

et sa vue :

```
def a_propos(request: HttpRequest) -> HttpResponse:
    return HttpResponse("BiblioCours est un projet d'apprentissage de Django.")
```

La route statique `a-propos/` précède la route dynamique `<slug:slug>/`. Si vous inversiez l'ordre, le mot `a-propos` serait un slug valide : la route dynamique le capturerait et la page « À propos » deviendrait inaccessible.

⚠️ Expérience à faire

Échangez temporairement l'ordre de `a-propos/` et de `<slug:slug>/` dans `cours/urls.py`, puis ouvrez `/cours/a-propos/`. Que voyez-vous ? Remettez ensuite l'ordre correct. Retenez la règle : **les routes les plus spécifiques d'abord, les routes génériques ensuite.**

Corollaire : un cours dont le slug vaudrait `a-propos` serait inaccessible. Nous reviendrons sur ce piège quand nous validerons les slugs (Section).

Fabriquer une URL : reverse et les noms de routes

Pour obtenir une URL depuis son nom, ouvrez le shell Django :

```
uv run python manage.py shell
```

```
from django.urls import reverse

reverse("cours:a_propos")
# "/cours/a-propos/"

reverse("cours:detail", kwargs={"slug": "python-pour-debutants"})
# "/cours/python-pour-debutants/"
```

Cette technique est préférable à `"/cours/a-propos/"` écrit en dur : si le préfixe `cours/` change dans `config/urls.py`, tous les liens suivent automatiquement. Elle sert notamment dans les redirections, les modèles (`get_absolute_url`) et les tests. Dans un template, l'équivalent est la balise `{% url 'cours:a_propos' %}`.

Le préfixe `cours:` (l'espace de noms défini par `app_name`) évite les collisions : si une autre app définissait aussi une route `accueil`, on distinguerait `cours:accueil` et `autre:accueil`.

Si vous donnez de mauvais arguments, Django lève `NoReverseMatch`. Essayez `reverse("cours:detail", kwargs={"pk": 1})` : le message d'erreur indique le nom des paramètres attendus. Apprenez à lire ces messages, ils sont précis.

Un premier test automatique

Nous allons vérifier nos routes par un test. `startapp` a créé un fichier `cours/tests.py`. Comme nous aurons plusieurs fichiers de tests, remplacez-le par un **paquet** :

```
rm cours/tests.py
mkdir cours/tests
touch cours/tests/__init__.py
```

(Un fichier `tests.py` et un dossier `tests/` ne peuvent pas coexister : Python ne saurait pas lequel importer.) Créez `cours/tests/test_urls.py` :

```
from django.test import SimpleTestCase
from django.urls import resolve, reverse

from cours import views

class RoutesTests(SimpleTestCase):
    def test_reverse_a_propos(self):
        self.assertEqual(reverse("cours:a_propos"), "/cours/a-propos/")

    def test_a_propos_repond_200(self):
        response = self.client.get(reverse("cours:a_propos"))

        self.assertEqual(response.status_code, 200)

    def test_a_propos_n_est_pas_capturee_par_le_detail(self):
        correspondance = resolve("/cours/a-propos/")

        self.assertEqual(correspondance.func, views.a_propos)
```

Lancez-les :

```
uv run python manage.py test
```

Trois éléments sont nouveaux : `self.client` simule un navigateur sans lancer de serveur ; `resolve` fait l'opération inverse de `reverse` (de l'URL vers la vue) ; `SimpleTestCase` est la classe de test la plus légère, qui **interdit** tout accès à la base de données. Le Section détaillera les autres classes de test. Le troisième test est précieux : il échouerait si quelqu'un inversait l'ordre des routes.

Requêtes et réponses : les distinctions importantes

Une requête contient plusieurs catégories d'information :

```
def rechercher(request):
    terme = request.GET.get("q", "")
    langue = request.headers.get("Accept-Language")
    utilisateur = request.user
    return HttpResponse(
        f"Recherche : {terme}",
        content_type="text/plain; charset=utf-8",
    )
```

Attribut	Contenu	Remarque
<code>request.method</code>	"GET", "POST"...	décide de l'action à mener
<code>request.GET</code>	paramètres de l'URL (<code>?q=django</code>)	données de lecture, saisies par l'utilisateur
<code>request.POST</code>	données d'un formulaire envoyé	saisies par l'utilisateur
<code>request.user</code>	utilisateur courant, éventuellement anonyme	fourni par un middleware
<code>request.headers</code>	en-têtes HTTP	décrivent le contexte, non fiables
<code>request.path</code>	chemin demandé	

Une vue ne doit pas faire confiance aux données reçues. Même un champ qui semble numérique ou une valeur provenant d'un navigateur doit être validé côté serveur. Les formulaires Django aideront à centraliser cette validation.

Remarquez le `content_type="text/plain"` dans l'exemple ci-dessus. Par défaut, une `HttpResponse` est du HTML : si l'on y recopiait `terme` tel quel et que l'utilisateur tapait `<script>...</script>`, le navigateur l'exécuterait (c'est une faille XSS, étudiée au Section). En déclarant la réponse comme du texte brut, le navigateur n'interprète rien. Les templates, eux, échappent automatiquement les variables.

Pour construire une réponse, `HttpResponse(contenu, status=200, content_type=...)` accepte aussi un statut explicite. Pour signaler une page inexistante, on lèvera plutôt `Http404` (`from django.http import Http404`), que Django transforme en réponse 404.

Premier modèle mental à retenir

Quand une page ne fonctionne pas, diagnostiquez dans cet ordre :

1. l'URL est-elle réellement atteinte ?
2. la vue est-elle appelée avec les bons paramètres ?
3. la requête vers la base renvoie-t-elle le résultat attendu ?
4. le contexte transmis au template porte-t-il le bon nom ?
5. le template affiche-t-il ce nom et cette valeur ?

6. le navigateur reçoit-il le HTML attendu ?

Cette méthode est plus rapide que de modifier simultanément le modèle, la vue, le template et le CSS. Deux outils aident déjà aux étapes 1 et 2 : le **journal du terminal** (la requête est-elle arrivée ? quel statut ?) et la **page 404 de débogage** de Django, qui, en mode **DEBUG**, liste toutes les routes qu'elle a essayées dans l'ordre.

Checkpoint

À ce stade, `cours/urls.py` contient trois routes et `cours/views.py` trois vues. Vérifiez :

- ☐ `/cours/` affiche « Bienvenue dans BiblioCours » ;
- ☐ `/cours/a-propos/` affiche la page « À propos » ;
- ☐ `/cours/python-pour-debutants/` affiche « Le cours demandé est : python-pour-debutants » ;
- ☐ `uv run python manage.py test` rapporte 3 tests réussis ;
- ☐ `uv run python manage.py check` ne signale aucun problème ;
- ☐ vous avez fait un commit (`feat: premières routes et vues`).

Erreurs fréquentes

Symptôme	Cause probable	Remède
404 sur <code>/cours/</code>	<code>include("cours.urls")</code> absent ou faute dans le préfixe	relire <code>config/urls.py</code>
<code>ModuleNotFoundError: cours.urls</code>	fichier <code>cours/urls.py</code> non créé	le créer
<code>NoReverseMatch</code>	nom de route, espace de noms ou argument incorrect	lire le message : il liste ce qui est attendu
<code>/cours/a-propos/</code> affiche « Le cours demandé est... »	route dynamique placée avant la route statique	inverser l'ordre
<code>TypeError: ... got an unexpected keyword argument</code>	nom du convertisseur différent du paramètre de la vue	aligner les deux noms
<code>Ran 0 tests</code>	<code>__init__.py</code> manquant ou fichier ne commençant pas par <code>test</code>	vérifier le paquet <code>cours/tests/</code>

À retenir

- Le chemin est : navigateur → routeur → vue → (modèle) → template → navigateur.
- Le routeur lit les motifs **dans l'ordre** et s'arrête au premier qui correspond : le spécifique avant le générique.
- `include` délègue, `app_name` isole, `name` identifie.
- On fabrique les URLs avec `reverse` (Python) ou `{% url %}` (template), jamais à la main.
- Les données d'une requête (GET, POST, en-têtes) viennent de l'extérieur : on les valide toujours.
- Un code de statut en `4xx` oriente vers le client, en `5xx` vers le serveur.

Exercices

1. Décrivez, dans l'ordre, tous les éléments traversés par la requête GET `/cours/python-pour-debutants/` dans BiblioCours, depuis le navigateur jusqu'à la réponse.

2. Pour chacun des éléments suivants, dites s'il relève du modèle, de la vue, du template ou du routeur : (a) l'en-tête HTML de la page ; (b) « un cours brouillon est invisible au public » ; (c) « `/cours/matiere/<slug>/` » ;
(d) le format de date affiché.
3. Donnez deux scénarios concrets où écrire `"/cours/12/"` en dur dans un template causerait des problèmes.
4. Avec les routes dans l'ordre `["<slug:slug>/", "a-propos/"]`, que renvoie `/cours/a-propos/` ? Justifiez, puis vérifiez expérimentalement.
5. Parmi `12/`, `-3/`, `abc/`, `012/`, lesquels correspondent au motif `<int:pk>/` ? Faites d'abord une prédiction, puis vérifiez-la après l'exercice 6, en appelant `resolve("/cours/exemple/12/")` (et les autres valeurs) dans le shell.
6. Ajoutez temporairement la route `exemple/<int:numero>/` qui renvoie le double de `numero`. Que se passe-t-il pour `/cours/exemple/abc/` ? Pourquoi la vue n'est-elle jamais appelée ? Supprimez ensuite la route.
7. Ajoutez la route `recherche/` (avant la route dynamique) et la vue `rechercher` de ce chapitre. Testez `/cours/recherche/?q=django` puis `/cours/recherche/` sans paramètre. Quel est le comportement dans le second cas, et pourquoi ?
8. Dans le shell, provoquez volontairement un `NoReverseMatch` avec `reverse("cours:detail", kwargs={"pk": 1})`. Lisez le message et corrigez l'appel.
9. Écrivez un test qui vérifie que `reverse("cours:detail", kwargs={"slug": "python"})` vaut `/cours/python/` et que `resolve("/cours/python/")` transmet bien `slug="python"` (indice : consultez l'attribut `kwargs` de la correspondance).
10. Avec `DEBUG = True`, ouvrez `/cours/a/b/c/` et lisez la page 404 de débogage. Quelles routes ont été essayées, et dans quel ordre ?
11. Créez une seconde app temporaire `demo` avec une route nommée `accueil`, incluse sous `demo/`. Vérifiez que `reverse("cours:accueil")` et `reverse("demo:accueil")` donnent des URLs différentes. Que se passerait-il sans espace de noms ? Supprimez ensuite l'app (ne la commitez pas).
12. Rédigez un tableau de toutes les routes de BiblioCours à ce stade (URL, nom, vue, méthode attendue). Gardez-le à jour au fil des chapitres : c'est une documentation précieuse.

Modéliser les données avec l'ORM et les migrations

i Objectifs d'apprentissage

À la fin de ce chapitre, vous saurez :

- traduire une description métier en modèles Django et en relation un-à-plusieurs ;
- choisir les types de champs et les options (`null`, `blank`, `unique`, `choices`, `on_delete...`) ;
- expliquer le rôle d'une migration et utiliser `makemigrations`, `migrate`, `showmigrations` et `sqlmigrate` ;
- créer, lire, filtrer, modifier et supprimer des objets avec l'ORM ;
- expliquer l'évaluation différée d'un `QuerySet` et repérer le problème N+1 ;
- distinguer `save()` et `update()`, et protéger un ensemble d'écritures avec une transaction ;
- écrire une règle de validation au niveau du modèle.

Où en sommes-nous ?

BiblioCours dispose de routes et de vues qui renvoient du texte. Il n'y a pas encore de données : le contenu est codé en dur dans les vues. Dans ce chapitre, nous décrivons les données (matières et cours), nous créons les tables correspondantes et nous apprenons à les manipuler depuis Python. À la fin, la base contiendra un jeu de données de démonstration dont nous nous servirons aux chapitres 4 et 5.

Penser d'abord au domaine

Avant d'écrire `models.py`, décrivons les objets du métier. BiblioCours contient des **matières** et des **cours**. Une matière regroupe plusieurs cours ; un cours appartient à une seule matière. Cette phrase donne déjà une relation « un-à-plusieurs » :

```
Matiere 1      < * Cours
(une matière a plusieurs cours ; un cours a une seule matière)
```

Un bon modèle répond à des questions concrètes :

- quelles informations doivent être obligatoires ?
- quelle taille maximale une valeur peut-elle avoir ?
- deux enregistrements peuvent-ils porter la même valeur ?
- que se passe-t-il si la matière d'un cours est supprimée ?
- quelle date représente la création et quelle date représente la publication ?

Le modèle n'est pas seulement un miroir de la table SQL. Il formalise des règles et fournit une interface Python pour manipuler les données. L'ORM (*Object-Relational Mapper*) de Django traduit cette interface en requêtes SQL adaptées à la base configurée ([Django Software Foundation 2025e](#)). La correspondance est systématique :

Côté Python	Côté base de données
une classe de modèle (<code>Cours</code>)	une table (<code>cours_cours</code>)

Côté Python	Côté base de données
un attribut (<code>titre</code>)	une colonne
une instance (<code>Cours(...)</code>)	une ligne
un <code>QuerySet</code>	une requête <code>SELECT</code>

Le nom de la table est formé du nom de l'app et du nom du modèle en minuscules : `cours` + `_` + `cours`.

Écrire les modèles de BiblioCours

Remplacez le contenu de `cours/models.py` par :

```
from django.core.exceptions import ValidationError
from django.db import models
from django.urls import reverse

class Matiere(models.Model):
    nom = models.CharField(max_length=100, unique=True)
    slug = models.SlugField(max_length=110, unique=True)
    description = models.TextField(blank=True)

    class Meta:
        ordering = ["nom"]
        verbose_name = "matière"
        verbose_name_plural = "matières"

    def __str__(self):
        return self.nom

class Cours(models.Model):
    class Statut(models.TextChoices):
        BROUILLON = "brouillon", "Brouillon"
        PUBLIE = "publie", "Publié"

    titre = models.CharField(max_length=200)
    slug = models.SlugField(max_length=220, unique=True)
    resume = models.CharField(max_length=500)
    contenu = models.TextField()
    statut = models.CharField(
        max_length=20,
        choices=Statut.choices,
        default=Statut.BROUILLON,
    )
    matiere = models.ForeignKey(
        Matiere,
        on_delete=models.PROTECT,
        related_name="cours",
    )
    date_creation = models.DateTimeField(auto_now_add=True)
    date_modification = models.DateTimeField(auto_now=True)
    date_publication = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-date_creation"]
```

```

        verbose_name = "cours"
        verbose_name_plural = "cours"
        indexes = [
            models.Index(fields=["statut", "-date_creation"]),
        ]

    def __str__(self):
        return self.titre

    def clean(self):
        super().clean()
        if self.statut == self.Statut.PUBLIE and self.date_publication is None:
            raise ValidationError(
                {
                    "date_publication": (
                        "Un cours publié doit avoir une date de publication."
                    )
                }
            )

    def get_absolute_url(self):
        return reverse("cours:detail", kwargs={"slug": self.slug})

    @property
    def est_publie(self):
        return self.statut == self.Statut.PUBLIE

```

Remarquez que nous n'avons déclaré aucune clé primaire : Django ajoute automatiquement une colonne `id` (un entier auto-incrémenté), que l'on retrouve aussi sous le nom `pk` (*primary key*). Le type de cette colonne est défini par `DEFAULT_AUTO_FIELD` dans `settings.py`.

Pourquoi ces champs ?

`CharField` représente une chaîne bornée. `max_length` est utile pour la base, les formulaires et la documentation de l'intention. `TextField` convient au contenu dont la taille est plus libre. `SlugField` représente une valeur adaptée aux URLs, généralement composée de lettres minuscules, chiffres et tirets.

⚠ `TextField(max_length=...)` ne protège pas la base

Pour un `TextField`, `max_length` est seulement transmis au widget de formulaire : ni le modèle ni la base ne l'imposent. C'est pourquoi le résumé est un `CharField(max_length=500)` : la limite y est réellement appliquée.

`unique=True` crée une contrainte d'unicité. Ce n'est pas seulement une validation d'interface : la base elle-même empêchera deux matières d'avoir le même slug. La base est la dernière barrière, car deux requêtes concurrentes peuvent passer simultanément une validation purement applicative.

`TextChoices` évite les chaînes dispersées dans le code. La valeur enregistrée est "publie" et le libellé affiché est "Publié". On peut écrire `Cours.Statut.PUBLIE` dans le code et laisser Django gérer les choix.

`auto_now_add` est évalué à la création ; `auto_now` à chaque sauvegarde.

La différence entre `null` et `blank` revient souvent et prête à confusion :

Option	Concerne	Effet
<code>null=True</code>	la base	la colonne accepte la valeur <code>NULL</code>

Option	Concerne	Effet
<code>blank=True</code>	la validation (formulaires, admin)	le champ peut rester vide
<code>default=...</code>	la création	valeur utilisée si rien n'est fourni

`blank=True` ne rend donc pas automatiquement une colonne SQL nullable. Ici, `date_publication` est `null=True`, `blank=True` : un brouillon n'a pas de date de publication. En revanche, pour un champ texte comme `description`, on met seulement `blank=True` : la valeur « vide » est la chaîne "", et non NULL (avoir deux façons de dire « vide » compliquerait les requêtes).

La classe `Meta` regroupe les options du modèle : `ordering` définit l'ordre par défaut des résultats, `verbose_name` les noms affichés dans l'administration (sans `verbose_name_plural = "cours"`, Django afficherait « courss »), et `indexes` un index de base de données qui accélère les requêtes filtrant sur `statut` et triant sur la date. La méthode `__str__` donne la représentation textuelle d'un objet, utilisée dans l'administration et le shell. `get_absolute_url` centralise l'adresse d'un objet : templates et redirections n'auront jamais à reconstruire l'URL. Enfin `est_publie` est une propriété Python : elle se calcule à partir de `statut` et n'est pas une colonne.

Choisir `on_delete`

La relation vers `Matiere` utilise `PROTECT`. Supprimer une matière qui possède encore des cours serait probablement une erreur métier ; Django lève donc une exception plutôt que d'effacer silencieusement le catalogue.

Les comportements courants sont :

- `CASCADE` : supprimer les objets liés ;
- `PROTECT` : refuser la suppression ;
- `SET_NULL` : mettre la relation à NULL si `null=True` ;
- `SET_DEFAULT` : utiliser la valeur par défaut ;
- `DO_NOTHING` : ne rien orchestrer, choix réservé aux cas maîtrisés.

Il n'existe pas de choix universel. Pour une photo liée à un profil, `CASCADE` peut être logique. Pour une facture liée à un client, `PROTECT` est souvent plus sûr. Posez-vous la question : *si le parent disparaît, l'enfant a-t-il encore un sens ?*

L'argument `related_name="cours"` nomme la relation **inverse** : à partir d'une matière, `matiere.cours` donne tous ses cours.

Valider au niveau du modèle

Notre méthode `clean()` exprime une règle métier : un cours publié doit avoir une date de publication. Cette règle appartient au modèle, car elle doit rester vraie quel que soit le chemin de création : formulaire public, administration ou script.

Mais attention : `save()` n'appelle pas `clean()`. Seul `full_clean()` lance toutes les validations (champs, `clean()`, unicité), et ce sont les `ModelForm` et l'administration qui l'appellent pour vous (chapitre 5). Dans le shell, un `Cours.objects.create(...)` écrit en base sans rien vérifier de ce qui n'est pas une contrainte SQL. Nous le constaterons plus bas.

Les migrations : l'historique du schéma

Un modèle Python et une table SQL ne sont pas la même chose. La **migration** est le fichier qui décrit une transformation du schéma. Le flux est toujours le même :

```
modèle modifié  makemigrations  fichier de migration  migrate  base mise à jour
```

```
uv run python manage.py makemigrations cours
uv run python manage.py migrate
```

`makemigrations` compare l'état actuel des modèles avec l'historique des migrations et génère un fichier Python dans `cours/migrations/`. `migrate` exécute les migrations non appliquées dans la base. Cette seconde commande crée le fichier `db.sqlite3` s'il n'existe pas et applique aussi les migrations des composants Django (authentification, administration, sessions) : l'avertissement rencontré au chapitre 1 disparaît.

Ouvrez `cours/migrations/0001_initial.py`. Vous y lirez, en version abrégée :

```
class Migration(migrations.Migration):
    initial = True

    dependencies = []

    operations = [
        migrations.CreateModel(
            name="Matiere",
            fields=[
                ("id", models.BigAutoField(primary_key=True, ...)),
                ("nom", models.CharField(max_length=100, unique=True)),
                # ...
            ],
        ),
        migrations.CreateModel(name="Cours", fields=[...]),
    ]
```

Une migration est du **code** : relisez-la avant de l'appliquer. Inspectez aussi ce que Django prévoit :

```
uv run python manage.py showmigrations
uv run python manage.py sqlmigrate cours 0001
```

`showmigrations` liste les migrations et indique par [X] celles qui sont appliquées. `sqlmigrate` montre le SQL généré sans l'exécuter :

```
BEGIN;
CREATE TABLE "cours_matiere" (
    "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
    "nom" varchar(100) NOT NULL UNIQUE,
    ...
);
```

C'est un excellent outil pour relier le code Python à la réalité de la base.

Faire évoluer un modèle

Quand vous modifiez un modèle (ajout d'un champ, par exemple), relancez `makemigrations` : Django produit une **nouvelle** migration (0002_...). Si le champ est obligatoire et que des lignes existent déjà, Django demande quelle valeur donner aux lignes existantes : fournissez un **default** plutôt que de laisser une colonne NOT NULL sans valeur.

⚠ Ne modifiez pas une migration déjà partagée

Une migration appliquée fait partie de l'histoire du schéma. En équipe, créez une nouvelle migration plutôt que de réécrire une ancienne. Modifier une migration déjà exécutée peut créer des divergences entre les bases des collaborateurs et la base de production.

Tant qu'une migration n'a **jamais** quitté votre machine (pas de commit partagé), vous pouvez la supprimer, effacer `db.sqlite3` et la régénérer : c'est une façon propre de corriger un modèle en cours d'apprentissage.

Les fichiers de migration doivent être versionnés. La base SQLite locale ne l'est généralement pas, mais les instructions permettant de la recréer le sont.

Créer et lire des objets dans le shell

```
uv run python manage.py shell
```

Créer

```
from django.utils import timezone

from cours.models import Cours, Matiere

informatique = Matiere.objects.create(
    nom="Informatique",
    slug="informatique",
    description="Comprendre les outils numériques.",
)
maths = Matiere.objects.create(nom="Mathématiques", slug="mathematiques")

python_cours = Cours.objects.create(
    titre="Python pour débutants",
    slug="python-pour-debutants",
    resume="Les bases du langage Python.",
    contenu="Variables, fonctions et modules.",
    matiere=informatique,
    statut=Cours.Statut.PUBLIE,
    date_publication=timezone.now(),
)

Cours.objects.create(
    titre="Algèbre linéaire",
    slug="algebre-lineaire",
    resume="Vecteurs, matrices et espaces vectoriels.",
    contenu="Brouillon en cours d'écriture.",
    matiere=maths,
) # statut par défaut : brouillon
```

Relancez la même commande `Matiere.objects.create(slug="informatique", ...)` : vous obtenez `IntegrityError: UNIQUE constraint failed`. C'est la contrainte `unique=True` qui protège la base.

Constatons maintenant que `create` ne valide pas la règle du modèle :

```
incoherent = Cours(
    titre="Cours incohérent",
    slug="cours-incoherent",
    resume="Publié sans date.",
    contenu="...",
    matiere=informatique,
    statut=Cours.Statut.PUBLIE,
)
incoherent.full_clean()  # lève ValidationError : date_publication est requise
incoherent.save()       # écrit quand même (aucune contrainte SQL n'est violée)
incoherent.delete()     # nettoie
```

Un jeu de données pour les chapitres suivants

Pour tester la pagination au chapitre 4, créons vingt-cinq cours de démonstration. `get_or_create` ne crée l'objet que s'il n'existe pas déjà : on peut donc relancer le script sans erreur.

```
maintenant = timezone.now()
for numero in range(1, 26):
    Cours.objects.get_or_create(
        slug=f"cours-de-demo-{numero}",
        defaults={
            "titre": f"Cours de démo {numero}",
            "resume": f"Résumé du cours de démonstration numéro {numero}.",
            "contenu": "Contenu de démonstration.",
            "matiere": informatique,
            "statut": Cours.Statut.PUBLIE,
            "date_publication": maintenant,
        },
    )
```

Lire et naviguer dans les relations

```
python_cours.matiere          # <Matiere: Informatique>
informatique.cours.all()      # tous les cours de la matière
Cours.objects.filter(statut=Cours.Statut.PUBLIE)
```

Chaque accès à `.objects` construit ou exécute un `QuerySet`. Un `QuerySet` est **évalué de manière différée** : écrire `cours = Cours.objects.filter(...)` ne signifie pas nécessairement qu'une requête SQL est déjà partie. L'évaluation arrive lors d'une itération, d'une conversion en liste, d'un `len`, ou de l'accès à certains résultats. Vous pouvez voir le SQL qui sera exécuté :

```
qs = Cours.objects.filter(statut=Cours.Statut.PUBLIE)  # aucune requête SQL
print(qs.query)                                       # affiche le SQL
len(qs)                                              # c'est ici qu'elle part
```

Cette paresse permet de **chaîner** les opérations (`filter`, `exclude`, `order_by`...) pour construire une seule requête optimisée.

Filtrer, trier et limiter

```
Cours.objects.filter(titre__icontains="python")
Cours.objects.filter(matiere__slug="informatique")
Cours.objects.filter(
    statut=Cours.Statut.PUBLIE,
    matiere__nom__iexact="informatique",
)
Cours.objects.exclude(statut=Cours.Statut.BROUILLON)
Cours.objects.order_by("-date_publication")[:10]
```

Les doubles underscores traversent des relations et expriment des recherches. Les plus utiles :

Suffixe	Signification	Exemple
<code>__exact</code> (défaut)	égalité	<code>statut="publie"</code>
<code>__iexact</code>	égalité sans casse	<code>nom__iexact="python"</code>
<code>__contains</code> / <code>__icontains</code>	contient (sans casse pour i)	<code>titre__icontains="py"</code>
<code>__startswith</code>	commence par	<code>titre__startswith="Intro"</code>
<code>__in</code>	appartient à une liste	<code>statut__in=["publie"]</code>
<code>__gt</code> , <code>__gte</code> , <code>__lt</code> , <code>__lte</code>	comparaisons	<code>date_publication__gte=...</code>
<code>__isnull</code>	est (ou non) nul	<code>date_publication__isnull=True</code>

Le tri `"-date_publication"` (avec un tiret) est décroissant ; `[:10]` limite à dix résultats (c'est un LIMIT SQL, pas un découpage en mémoire).

`filter` renvoie un `QuerySet` (éventuellement vide), tandis que `get` attend **exactement un** résultat :

```
cours = Cours.objects.get(slug="python-pour-debutants")
```

Si aucun objet ne correspond, `get` lève `Cours.DoesNotExist` ; si plusieurs correspondent, il lève `Cours.MultipleObjectsReturned`. Pour une vue, `get_object_or_404` transforme l'absence en réponse HTTP 404, plutôt que de laisser remonter une exception non gérée. D'autres méthodes pratiques : `count()`, `exists()`, `first()`.

select_related et prefetch_related

Afficher une liste de cours et le nom de leur matière peut provoquer une requête par cours si l'on ne prépare pas la relation. C'est le problème classique du **N+1** : une requête pour la liste, puis une de plus pour chacun des N éléments. Mesurons-le :

```
from django.db import connection
from django.test.utils import CaptureQueriesContext

with CaptureQueriesContext(connection) as sans:
    for cours in Cours.objects.all():
        cours.matiere.nom

with CaptureQueriesContext(connection) as avec:
    for cours in Cours.objects.select_related("matiere"):
        cours.matiere.nom

print(len(sans), len(avec))    # par exemple : 28 1
```

La version naïve exécute une requête par cours ; avec `select_related`, une seule jointure suffit. `select_related` utilise une jointure SQL pour les relations à valeur unique comme `ForeignKey`. Pour une relation inverse ou plusieurs-à-plusieurs, utilisez `prefetch_related`, qui exécute une requête supplémentaire pour toutes les relations et assemble les résultats en Python :

```
matieres = Matiere.objects.prefetch_related("cours").all()
```

Il ne faut pas ajouter ces méthodes mécaniquement. Mesurez les requêtes et choisissez l'outil adapté à la relation et à l'écran.

Modifier les données avec prudence

Ces deux formes ne sont pas équivalentes :

```
cours = Cours.objects.get(pk=1)
cours.titre = "Nouveau titre"
cours.save(update_fields=["titre", "date_modification"])
```

et :

```
Cours.objects.filter(pk=1).update(titre="Nouveau titre")
```

	<code>save()</code> sur une instance	<code>update()</code> sur un <code>QuerySet</code>
exécute	un <code>UPDATE</code> pour cet objet	un <code>UPDATE</code> pour toutes les lignes visées
appelle un <code>save()</code> personnalisé	oui	non
met à jour <code>auto_now</code>	oui	non (à fournir soi-même)
lance <code>clean()</code>	non	non

`save` passe par le cycle de sauvegarde de l'instance (d'où `update_fields=[..., "date_modification"]` : on demande explicitement de mettre à jour la date). `update` exécute une mise à jour SQL directe. Pour une opération en masse, `update` est efficace, mais il faut connaître ses conséquences : notamment qu'il laisse `date_modification` inchangé.

Pour une opération composée de plusieurs écritures, utilisez une transaction :

```
from django.db import transaction
from django.utils import timezone

@transaction.atomic
def publier(cours):
    cours.statut = Cours.Statut.PUBLIE
    cours.date_publication = timezone.now()
    cours.save(
        update_fields=["statut", "date_publication", "date_modification"]
    )
```

Si une exception survient dans le bloc atomique, Django annule les opérations de la transaction : soit toutes les écritures réussissent, soit aucune. La transaction protège la cohérence des données ; elle ne remplace pas la validation métier.

Concevoir avant de migrer

Ajoutez les champs uniquement lorsqu'ils correspondent à un besoin. Un champ `description` et un champ `contenu` ne sont pas interchangeables si leurs contraintes et leurs usages diffèrent. À l'inverse, ne créez pas un champ `est_publie` en plus de `statut` : deux sources de vérité peuvent diverger. C'est pourquoi `est_publie` est une propriété calculée, et non une colonne.

Le bon flux est :

1. décrire la règle métier ;
2. choisir le type et les contraintes ;
3. modifier le modèle ;
4. lire la migration générée ;
5. appliquer et tester la migration ;
6. essayer les cas normaux et les cas limites.

Checkpoint

- ☐ `uv run python manage.py makemigrations --check` ne signale aucun changement en attente ;
- ☐ `showmigrations` montre toutes les migrations de `cours` avec `[X]` ;
- ☐ `Cours.objects.count()` renvoie 27 dans le shell (1 cours Python, 1 brouillon, 25 cours de démo) ;
- ☐ `Cours.objects.filter(statut="publie").count()` renvoie 26 ;
- ☐ `matiere.delete()` échoue avec `ProtectedError` pour `informatique` ;
- ☐ vous avez fait un commit contenant `models.py` et `migrations/0001_initial.py`.

Erreurs fréquentes

Symptôme	Cause probable	Remède
<code>OperationalError: no such table: cours_cours</code>	<code>migrate</code> oublié	lancer <code>migrate</code>
<code>IntegrityError: UNIQUE constraint failed</code>	slug ou nom déjà utilisé	choisir une autre valeur
<code>IntegrityError: NOT NULL constraint failed</code>	champ obligatoire non renseigné	fournir une valeur ou un <code>default</code>
<code>ProtectedError</code>	suppression d'une matière qui a des cours	supprimer/déplacer d'abord les cours
<code>Cours.DoesNotExist</code>	<code>get</code> sans résultat	utiliser <code>filter</code> ou <code>get_object_or_404</code>
<code>Your models have changes that are not yet reflected...</code>	<code>makemigrations</code> oublié	le lancer, puis <code>migrate</code>
<code>create</code> accepte des données invalides	<code>create</code> ne valide pas	appeler <code>full_clean()</code> ou passer par un formulaire

À retenir

- Le modèle décrit des **règles**, pas seulement des colonnes ; la base est la dernière barrière.
- `null` concerne la base, `blank` la validation.
- Choisissez `on_delete` en vous demandant si l'enfant a un sens sans le parent.
- Une migration est un fichier de code qui raconte l'histoire du schéma ; on la relit et on la versionne.
- Un `QuerySet` est paresseux ; mesurez les requêtes pour repérer le N+1.
- `save()` et `update()` n'ont pas les mêmes effets ; `save()` ne valide pas.
- Une règle métier ne doit avoir qu'une seule source de vérité.

Exercices

1. Pourquoi déconseille-t-on `CharField(blank=True, null=True)` pour un champ texte ? Combien de valeurs « vides » cela autorise-t-il ?
2. Après `Cours.objects.filter(pk=1).update(titre="X")`, que vaut `date_modification` ? Et après un `save()` ? Justifiez.
3. Quel `on_delete` choisiriez-vous pour : (a) un commentaire lié à un cours ;
(b) un cours lié à son auteur, si l'on veut conserver le cours quand le compte disparaît ; (c) une commande liée à un client ?
4. Pourquoi `est_publie` est-elle une propriété et non un champ booléen ?
5. Combien de requêtes SQL chaque extrait déclenche-t-il, pour N cours ?

- (A) `for c in Cours.objects.all(): print(c.titre);`
- (B) `for c in Cours.objects.all(): print(c.matiere.nom);`
- (C) `for c in Cours.objects.select_related("matiere"): print(c.matiere.nom);`
- (D) `for m in Matiere.objects.all(): print(m.cours.count());`

6. Ajoutez un champ `niveau` avec trois choix (débutant, intermédiaire, avancé) et une valeur par défaut. Lancez `makemigrations`, lisez le fichier `0002_...` et le SQL (`sqlmigrate`). Que se passe-t-il pour les cours déjà existants ?
7. Ajoutez à `Cours.clean()` la règle « un cours publié doit avoir un résumé d'au moins vingt caractères ». Vérifiez-la avec `full_clean()` dans le shell, puis constatez que `create()` ne la déclenche pas.
8. Écrivez une requête qui retourne les cinq cours publiés les plus récents d'une matière donnée, triés par date de publication décroissante.
9. Comparez le nombre de requêtes d'une boucle avec et sans `select_related("matiere")` à l'aide de `CaptureQueriesContext`.

Indice — afficher le SQL dans le terminal : pour voir toutes les requêtes en développement, ajoutez dans `settings.py` un logger `django.db.backends` au niveau `DEBUG` avec un handler `StreamHandler` (la structure `LOGGING` est détaillée au Section). Les requêtes n'apparaissent que si `DEBUG = True`.

10. Testez `publier` : écrivez une variante qui lève une exception après `cours.save()`, à l'intérieur de `@transaction.atomic`. Relisez le cours depuis la base. Le statut a-t-il changé ? Que démontre ce résultat ?
11. Créez un modèle `Commentaire` (texte, date, clé étrangère vers `Cours` avec `CASCADE` et `related_name="commentaires"`). Migrez, créez quelques commentaires, puis supprimez un cours : que deviennent les commentaires ? Comparez avec le comportement de `PROTECT` sur `Matiere`. (Faites ce travail dans une branche Git : nous n'utiliserons pas `Commentaire` dans la suite.)
12. Imaginez un modèle `Tag` en relation plusieurs-à-plusieurs avec `Cours`. Dessinez sur papier les tables nécessaires, puis vérifiez votre schéma avec `sqlmigrate` : quelle table intermédiaire Django crée-t-il ? Quelle méthode utiliseriez-vous pour afficher tous les tags d'une liste de cours sans N+1 ?

Construire les pages : vues, URLs et templates

i Objectifs d'apprentissage

À la fin de ce chapitre, vous saurez :

- écrire des vues de liste et de détail qui interrogent la base ;
- lire et écrire du code dans le langage de templates de Django (variables, balises, filtres) ;
- organiser des templates avec un squelette parent, des enfants et des fragments réutilisables ;
- paginer une liste et ajouter une recherche par paramètre GET ;
- expliquer l'échappement automatique et pourquoi éviter `|safe` ;
- diagnostiquer une page vide ou une erreur de template.

Où en sommes-nous ?

Nous avons des routes (chapitre 2) et des données (chapitre 3) : 26 cours publiés, 1 brouillon et 2 matières. Mais les vues répondent encore par du texte brut. Nous allons maintenant remplacer la vue provisoire `accueil` par une vraie liste de cours, transformer `detail_cours` en page de détail, et produire du HTML avec des templates.

De la donnée à une page HTML

Une page dynamique ne devrait pas construire son HTML par concaténation de chaînes Python. La vue prépare les données et choisit le template ; le template décrit la représentation HTML. Cette séparation rend le HTML lisible par une personne qui travaille sur l'interface sans lui demander de modifier la logique métier.

Pour BiblioCours, nous voulons :

<code>/cours/</code>	liste des cours publiés
<code>/cours/<slug>/</code>	détail d'un cours
<code>/cours/matiere/<slug>/</code>	cours d'une matière
<code>/cours/a-propos/</code>	page statique (déjà créée)

Les noms de routes restent stables même si leur forme évolue. C'est pourquoi nous avons défini `app_name = "cours"` et des noms explicites ([Django Software Foundation 2025h](#)).

Le langage de templates en bref

Un template est un fichier HTML enrichi de trois constructions, qui ne s'exécutent jamais côté navigateur : Django les évalue **avant** d'envoyer la page.

Syntaxe	Rôle	Exemple
<code>{ variable }</code>	afficher une valeur	<code>{ cours.titre }</code>
<code>{% balise %}</code>	exécuter une logique	<code>{% for ... %}, {% if ... %}, {% url ... %}</code>

Syntaxe	Rôle	Exemple
<code>{ valeur\ filtre }</code>	transformer une valeur à l'affichage	<code>{ cours.date_publication\ date:"d/m/Y" }</code>
<code>{# ... #}</code>	commentaire invisible dans le HTML	<code>{# à revoir #}</code>

La notation pointée `cours.matiere.nom` est polyvalente : Django essaie successivement une clé de dictionnaire, un attribut, puis un appel de méthode sans argument (ce qui explique que `{ cours.get_absolute_url }` fonctionne sans parenthèses). Les balises les plus utiles de ce chapitre :

Balise	Usage
<code>{% extends "..." %}</code>	hériter d'un template parent (doit être la première balise)
<code>{% block nom %}...{% endblock %}</code>	définir ou remplacer une zone
<code>{% include "..." %}</code>	insérer un fragment réutilisable
<code>{% for x in liste %}...{% empty %}...{% endfor %}</code>	boucler, avec un cas « liste vide »
<code>{% if %}...{% elif %}...{% else %}...{% endif %}</code>	condition
<code>{% url 'app:nom' arg %}</code>	générer une URL à partir du nom d'une route
<code>{% load static %} / {% static 'chemin' %}</code>	adresse d'un fichier statique

Quelques filtres fréquents : `date` (formater une date), `linebreaks` (retours à la ligne vers paragraphes), `truncatewords:20` (couper un texte), `default:"-"` (valeur de repli), `length` (longueur), `pluralize` (« s » selon le nombre).

⚠ Les templates échouent en silence

Une variable inconnue, une faute de frappe dans un nom d'attribut ou une clé absente du contexte ne provoquent **pas** d'erreur : Django affiche une chaîne vide. C'est pratique pour un site en production, mais piégeux pour apprendre : une page « vide » est souvent le signe d'un nom mal orthographié. Nous reviendrons sur le diagnostic en fin de chapitre.

Les vues de liste et de détail

Les routes

Complétez `cours/urls.py`. La vue provisoire `accueil` disparaît : la liste des cours devient la page d'accueil de l'app.

```
from django.urls import path

from . import views

app_name = "cours"

urlpatterns = [
    path("", views.liste_cours, name="liste"),
    path("a-propos/", views.a_propos, name="a_propos"),
    path("matiere/<slug:slug>/", views.cours_par_matiere, name="par_matiere"),
    path("<slug:slug>/", views.detail_cours, name="detail"),
]
```

L'ordre mérite attention. La route statique `a-propos/` doit apparaître avant `<slug:slug>/` : sinon le mot `a-propos` serait capturé comme le slug d'un cours (voir Section). La route `matiere/<slug:slug>/` compte deux segments : le convertisseur `slug` n'acceptant pas de `/`, elle ne peut pas entrer en conflit avec `<slug:slug>/`. Nous la plaçons pourtant avant, par principe : « le plus spécifique d'abord » est une habitude qui évite les surprises quand le routeur grandit.

Les vues

Voici `cours/views.py` complet à ce stade :

```
from django.core.paginator import Paginator
from django.http import HttpRequest, HttpResponse
from django.shortcuts import get_object_or_404, render

from .models import Cours, Matiere

COURS_PAR_PAGE = 10

def _cours_publies():
    """QuerySet de base : les cours publiés, avec leur matière préchargée."""
    return Cours.objects.filter(statut=Cours.Statut.PUBLIE).select_related(
        "matiere"
    )

def _paginer(request, queryset):
    """Découpe un QuerySet en pages et renvoie la page demandée."""
    paginator = Paginator(queryset, COURS_PAR_PAGE)
    return paginator.get_page(request.GET.get("page"))

def a_propos(request: HttpRequest) -> HttpResponse:
    return HttpResponse("BiblioCours est un projet d'apprentissage de Django.")

def liste_cours(request):
    """Affiche les cours publiés, dix par page."""
    page_obj = _paginer(request, _cours_publies())
    return render(request, "cours/liste.html", {"page_obj": page_obj})

def detail_cours(request, slug):
    cours = get_object_or_404(_cours_publies(), slug=slug)
    return render(request, "cours/detail.html", {"cours": cours})

def cours_par_matiere(request, slug):
    matiere = get_object_or_404(Matiere, slug=slug)
    page_obj = _paginer(request, _cours_publies().filter(matiere=matiere))
    return render(
        request,
        "cours/liste.html",
        {"page_obj": page_obj, "matiere": matiere},
    )
```

Deux fonctions utilitaires (préfixées par `_`, convention Python pour un usage interne) évitent de répéter le filtre « cours publiés » et la pagination dans chaque vue. Si la règle de publication change, on la

modifie à un seul endroit. Remarquez aussi que `cours_par_matiere` passe **un objet Page** au template, exactement comme `liste_cours` : les deux pages partagent le même template.

`Paginator` découpe un `QuerySet` en pages de `COURS_PAR_PAGE` éléments ; `get_page` est tolérant : une page absente, négative ou non numérique donne la première ou la dernière page plutôt qu'une erreur.

Pourquoi render ?

`render(request, template_name, context) :`

1. charge le template ;
2. lui transmet un dictionnaire de contexte ;
3. produit une réponse HTTP avec le HTML.

La clé `"page_obj"` est un **contrat** entre la vue et le template. Si la vue emploie `"cours"` mais que le template parcourt `page_obj`, la page semble vide sans que la base soit nécessairement en cause.

404 plutôt que page vide

`get_object_or_404` exprime qu'un objet demandé doit exister. Une URL inconnue renvoie alors un statut 404 Not Found, ce qui est correct pour le navigateur, les moteurs de recherche et les tests. Remarquez aussi que `detail_cours` cherche **parmi les cours publiés uniquement** : un brouillon renvoie la même 404 qu'un cours inexistant, ce qui ne révèle même pas son existence. Il ne faut pas attraper toutes les exceptions et renvoyer une page générique : cela masque les erreurs de programmation et transforme parfois un 404 en 500.

Organiser les templates

Créez :

```
cours/
  static/
    cours/
      site.css
  templates/
    cours/
      base.html
      liste.html
      detail.html
      partials/
        carte_cours.html
```

Avec `APP_DIRS = True`, Django cherche un dossier `templates` dans les apps installées. Le sous-dossier `cours/` (on répète le nom de l'app) évite les collisions : si deux apps possédaient chacune un fichier `liste.html`, Django prendrait le premier trouvé. C'est pourquoi on référence toujours `"cours/liste.html"`.

La logique d'héritage est la suivante :

```
base.html      ← squelette : <html>, en-tête, pied de page, blocs
liste.html     ← remplit les blocs `title` et `content`
                (include) partials/carte_cours.html
detail.html    ← remplit les blocs `title` et `content`
```

Le template parent

cours/templates/cours/base.html :

```
{% load static %}
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}BiblioCours{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'cours/site.css' %}">
</head>
<body>
  <header class="site-header">
    <nav aria-label="Navigation principale">
      <a href="{% url 'cours:liste' %}">BiblioCours</a>
      <a href="{% url 'cours:liste' %}">Tous les cours</a>
      <a href="{% url 'cours:a_propos' %}">À propos</a>
    </nav>
  </header>

  <main class="container">
    {% block content %}{% endblock %}
  </main>

  <footer class="site-footer">
    <p>BiblioCours - apprendre en construisant.</p>
  </footer>
</body>
</html>
```

{% extends %} et les blocs permettent aux pages enfants de réutiliser le document entier. `block title` et `block content` sont des points d'extension. Cette technique applique le principe DRY (*Don't Repeat Yourself*) sans rendre toutes les pages identiques.

La feuille de style

Le template charge `cours/site.css` avec `{% static %}`. Les fichiers statiques placés dans le dossier `static/` d'une app sont trouvés automatiquement par Django en développement. Créez `cours/static/cours/site.css` :

```
:root { --principale: #1f4e79; --texte: #222; --fond: #fafafa; }
* { box-sizing: border-box; }
body {
  margin: 0; font-family: system-ui, sans-serif;
  color: var(--texte); background: var(--fond); line-height: 1.6;
}
.container { max-width: 48rem; margin: 0 auto; padding: 1rem; }
.site-header, .site-footer { background: var(--principale); color: #fff; padding: .75rem 1rem; }
.site-header a, .site-footer a { color: #fff; margin-right: 1rem; }
.course-card { background: #fff; border: 1px solid #ddd; border-radius: .5rem; padding: 1rem; margin: .5rem 0; }
.eyebrow { text-transform: uppercase; font-size: .75rem; letter-spacing: .05em; color: #666; margin: 0 0 .5rem 0; }
.meta { color: #666; font-size: .875rem; }
.pagination { display: flex; gap: 1rem; align-items: center; }
.messages .message { padding: .5rem 1rem; border-radius: .25rem; background: #e8f1fb; }
```

```
.message-success { background: #e6f4ea; }
.message-error { background: #fdecea; }
.errors, .error { color: #b00020; }
.form-field { margin-bottom: 1rem; }
.form-field input, .form-field textarea, .form-field select { width: 100%; padding: .5rem; }
.inline-form { display: inline; }
```

Ce CSS volontairement sobre sera aussi utilisé par les formulaires et les messages des chapitres suivants.
La configuration complète des fichiers statiques est expliquée au Section .

Afficher une collection avec liste.html

```
{% extends "cours/base.html" %}

{% block title %}
    {% if matiere %}Cours de {{ matiere.nom }}{% else %}Tous les cours{% endif %}
    - BiblioCours
{% endblock %}

{% block content %}
    {% if matiere %}
        <header class="page-heading">
            <p class="eyebrow">Matière</p>
            <h1>{{ matiere.nom }}</h1>
            <p>{{ matiere.description }}</p>
        </header>
    {% else %}
        <header class="page-heading">
            <p class="eyebrow">Catalogue</p>
            <h1>Tous les cours</h1>
        </header>
    {% endif %}

    <section aria-labelledby="cours-heading">
        <h2 id="cours-heading">Cours disponibles</h2>
        {% for cours in page_obj %}
            {% include "cours/partials/carte_cours.html" %}
        {% empty %}
            <p>Aucun cours publié pour le moment.</p>
        {% endfor %}
    </section>

    {% if page_obj.has_other_pages %}
        <nav aria-label="Pagination" class="pagination">
            {% if page_obj.has_previous %}
                <a href="?page={{ page_obj.previous_page_number }}">Précédent</a>
            {% endif %}
            <span>Page {{ page_obj.number }} sur {{ page_obj.paginator.num_pages }}</span>
            {% if page_obj.has_next %}
                <a href="?page={{ page_obj.next_page_number }}">Suivant</a>
            {% endif %}
        </nav>
    {% endif %}
{% endblock %}
```

Ce template sert à deux vues : quand `matiere` est présent dans le contexte, il affiche l'en-tête de la matière ; sinon l'en-tête du catalogue. Une variable absente du contexte vaut « faux » dans un `{% if %}`.

Le bloc `{% empty %}` évite une page silencieuse quand le QuerySet ne contient aucun élément. Les attributs `has_previous` et `has_next` appartiennent à l'objet `Page` produit par `Paginator`. Le lien `?page=2` est un lien **relatif** : il garde le chemin courant et ne change que le paramètre.

Le partial `partials/carte_cours.html` :

```
<article class="course-card">
  <p class="eyebrow">{{ cours.matiere.nom }}</p>
  <h3>
    <a href="{{ cours.get_absolute_url }}">{{ cours.titre }}</a>
  </h3>
  <p>{{ cours.resume }}</p>
  <p class="meta">
    Publié le {{ cours.date_publication|date:"d/m/Y" }}
  </p>
</article>
```

`include` rend un fragment réutilisable dans le contexte courant : il voit donc la variable de boucle `cours`. Le modèle fournit `get_absolute_url`, ce qui centralise la construction du lien. Et grâce à `select_related("matiere")` dans la vue, afficher `cours.matiere.nom` pour chaque carte ne coûte aucune requête supplémentaire.

Afficher le détail

`cours/templates/cours/detail.html` :

```
{% extends "cours/base.html" %}

{% block title %}{{ cours.titre }} - BiblioCours{% endblock %}

{% block content %}
  <article class="course-detail">
    <p class="eyebrow">
      <a href="{% url 'cours:par_matiere' cours.matiere.slug %}">
        {{ cours.matiere.nom }}
      </a>
    </p>
    <h1>{{ cours.titre }}</h1>
    <p class="meta">
      Mis à jour le {{ cours.date_modification|date:"d/m/Y à H:i" }}
    </p>
    <p class="lead">{{ cours.resume }}</p>
    <div class="course-content">
      {{ cours.contenu|linebreaks }}
    </div>
  </article>
{% endblock %}
```

! `{% load %}` n'est pas hérité

Un template enfant n'hérite pas des `{% load %}` de son parent. Si `detail.html` utilisait `{% static %}`, il devrait contenir lui-même `{% load static %}` (juste après `{% extends %}`). L'erreur typique est `Invalid block tag on line 4: 'static'.`

Les variables sont **échappées automatiquement**. C'est un mécanisme essentiel : si le contenu contient `<script>`, Django l'affiche comme du texte au lieu de l'exécuter. Concrètement, `<` devient `<` dans le HTML envoyé. Le filtre `linebreaks` transforme des retours à la ligne en paragraphes et éléments `<br>`, et échappe le texte avant de le faire.

N'utilisez pas `|safe` simplement pour « faire marcher » un affichage. Il désactive l'échappement et doit être réservé à du HTML produit par une source de confiance, nettoyé avec une stratégie explicite. La sécurité des templates est expliquée au Section .

Rechercher avec un paramètre GET

Ajoutons une recherche simple. **Remplacez** la vue `liste_cours` par :

```
from django.db.models import Q

def liste_cours(request):
    """Affiche les cours publiés, filtrés par le terme `q` s'il est fourni."""
    terme = request.GET.get("q", "").strip()
    queryset = _cours_publies()
    if terme:
        queryset = queryset.filter(
            Q(titre__icontains=terme) | Q(resume__icontains=terme)
        )
    contexte = {"page_obj": _paginer(request, queryset), "terme": terme}
    return render(request, "cours/liste.html", contexte)
```

(Placez l'import avec les autres, en haut du fichier.) Les arguments multiples de `filter(a=1, b=2)` sont combinés par **ET**. Pour exprimer un **OU**, on utilise des objets `Q` combinés avec `|` ; `&` donne un ET et `~` un NON. Ici : le terme apparaît dans le titre **ou** dans le résumé.

Les paramètres GET sont des chaînes fournies par l'utilisateur ; `icontains` protège contre l'injection SQL parce que l'ORM paramètre la requête, mais il faut tout de même limiter les coûts et la longueur dans une application publique.

Le formulaire de recherche se place dans `liste.html`, juste avant la section des cours :

```
<form method="get" role="search">
  <label for="q">Rechercher un cours</label>
  <input id="q" name="q" value="{{ terme }}" type="search">
  <button type="submit">Rechercher</button>
</form>
```

Un formulaire `method="get"` ajoute ses champs à l'URL : valider avec « python » mène à `/cours/?q=python`. Remarquez `value="{{ terme }}"` : l'échappement automatique protège cet attribut, même si l'utilisateur tape des guillemets ou du HTML.

Lire un bug de template

Pour une page vide, ne commencez pas par changer le CSS. Vérifiez, dans le shell ou dans la vue :

```
print(queryset.query)
print(queryset.count())
```

Puis inspectez le contrat :

- le contexte contient-il `page_obj` ?
- la boucle utilise-t-elle `page_obj` ?
- le filtre `statut` exclut-il vos données de test ?
- la route pointe-t-elle vers la bonne vue ?
- le fichier est-il bien sous `templates/cours/` ?

Deux outils aident à voir ce que le template reçoit : la balise `{% debug %}` (qui affiche tout le contexte et les modules importés) et, dans une vue, `print(contexte)`. Le mode `DEBUG` fournit aussi des pages d'erreur détaillées, mais il ne faut jamais en laisser l'affichage actif en production.

Checkpoint

Lancez le serveur et vérifiez :

- ☐ `/cours/` affiche dix cours et « Page 1 sur 3 » ; le lien « Suivant » fonctionne ;
- ☐ `/cours/cours-de-demo-1/` affiche le détail avec le lien de la matière ;
- ☐ `/cours/algebre-lineaire/` (un brouillon) renvoie **404** ;
- ☐ `/cours/matiere/informatique/` liste les cours de la matière ;
- ☐ `/cours/matiere/mathematiques/` affiche « Aucun cours publié » (son seul cours est un brouillon) ;
- ☐ `/cours/matiere/inconnue/` renvoie 404 ;
- ☐ `/cours/?q=python` ne montre que « Python pour débutants » ;
- ☐ la page est stylée (le CSS est chargé : onglet *Réseau* du navigateur, statut 200) ;
- ☐ `uv run python manage.py test` passe toujours.

Erreurs fréquentes

Symptôme	Cause probable	Remède
<code>TemplateDoesNotExist:</code> <code>cours/liste.html</code>	mauvais dossier ou nom	vérifier <code>cours/templates/cours/liste.html</code> ; la page de debug liste les chemins explorés
page de liste vide alors que la base contient des cours	nom du contexte différent de celui du template, ou filtre trop strict	comparer clé de contexte et variable ; <code>print(queryset.count())</code>
<code>NoReverseMatch</code> dans <code>{% url %}</code>	mauvais nom ou mauvais nombre d'arguments	vérifier <code>app_name</code> , <code>name</code> et les arguments
<code>Invalid block tag ...</code> 'static'	<code>{% load static %}</code> manquant	l'ajouter dans le template qui utilise <code>{% static %}</code>
<code>TemplateSyntaxError:</code> <code>Unclosed tag</code>	<code>{% endif %}</code> , <code>{% endfor %}</code> ou <code>{% endblock %}</code> oublié	refermer chaque balise ouverte
CSS non appliqué	fichier mal placé (404 sur <code>/static/...</code>)	vérifier <code>cours/static/cours/site.css</code>
la page 2 d'une recherche ignore le terme	le lien de pagination ne contient pas <code>q</code>	voir l'exercice 7
<code>UnorderedObjectListWarning</code>	pagination d'un <code>QuerySet</code> non trié	garder <code>ordering</code> dans <code>Meta</code>

À retenir

- La vue **prépare** (données, contexte) ; le template **présente** (HTML).
- Le contexte est un contrat : une clé mal nommée donne une page vide, sans erreur.
- Les routes spécifiques précèdent les routes génériques.

- `extends/block` factorisent le squelette; `include` factorise un fragment; `{% load %}` n'est pas hérité.
- Les variables sont échappées automatiquement; `|safe` désactive cette protection.
- Un objet absent ou non publié se traduit par une 404, pas par une page vide.
- Utilisez `select_related` quand un template parcourt une relation.

Exercices

1. Dessinez l'arbre d'héritage de `liste.html` : quel template le parent fournit-il, quels blocs sont remplacés, quel fragment est inclus ?
2. Qu'affiche Django pour `{ cours.auteur }` si le modèle n'a pas de champ `auteur` ? Pourquoi est-ce un piège ?
3. Pourquoi `detail_cours` renvoie-t-il une 404 pour un brouillon plutôt qu'une erreur « accès refusé » (403) ?
4. Quelle est la différence entre `{ cours.contenu }` et `{ cours.contenu|safe }` si le contenu vaut `<b>Bonjour</b>` ? Dans quel cas le second est-il dangereux ?
5. Pourquoi le template de la liste utilise-t-il `{% empty %}` plutôt qu'un `{% if %}` séparé ?
6. Ajoutez sur chaque carte un lien « Voir la matière » qui mène à la liste des cours de cette matière (utilisez `{% url %}`).
7. **Corrigez un bug.** Avec `COURS_PAR_PAGE = 2`, cherchez `python`, puis cliquez sur « Suivant » : que devient le terme de recherche ? Corrigez les liens de pagination pour qu'ils conservent `q`.

Indice : un lien peut contenir plusieurs paramètres : `?page=2&q=python`. Utilisez `{ terme|urlencode }` pour protéger les caractères spéciaux.

8. Créez une page 404 personnalisée avec un template `404.html` (à la racine d'un dossier de templates) étendant `cours/base.html`. Vérifiez qu'elle s'affiche avec `DEBUG = False`.
Indice : avec `DEBUG = True`, Django affiche toujours sa page de debug. Pour tester, basculez temporairement `DEBUG = False` et ajoutez `"localhost"` à `ALLOWED_HOSTS`. Le dossier `templates/` global sera configuré au Section ; en attendant, placez le fichier dans `cours/templates/404.html`.
9. Transformez la page « À propos » : remplacez `HttpResponse` par un vrai template `cours/a_propos.html` étendant `base.html`. La route et son nom ne doivent pas changer.
10. Utilisez le filtre `truncatewords` pour n'afficher que les 25 premiers mots du résumé dans les cartes. Que se passe-t-il pour un résumé plus court ?
11. Ajoutez une vue et une page `/cours/matieres/` qui liste toutes les matières avec le nombre de cours publiés de chacune. Évitez le problème N+1 (indice : `annotate` et `Count` avec un filtre, ou `prefetch_related`). Pensez à l'ordre des routes !
12. Ajoutez un tri `?tri=recent` / `?tri=titre` à la liste. Quelles valeurs de `tri` acceptez-vous ? Que faites-vous d'une valeur inconnue ? Pourquoi ne faut-il jamais passer `request.GET["tri"]` directement à `order_by` ?

Saisir et administrer les données : formulaires et admin

Objectifs d'apprentissage

À la fin de ce chapitre, vous saurez :

- expliquer pourquoi la validation doit se faire côté serveur et décrire le cycle liaison → validation → action ;
- écrire un `ModelForm` avec validation par champ et validation croisée ;
- implémenter les vues de création, de modification et de suppression selon le schéma POST/Redirect/GET ;
- rendre un formulaire dans un template, avec ses erreurs et son jeton CSRF ;
- configurer l'administration Django (liste, filtres, recherche, actions) ;
- afficher des messages utilisateur après une action.

Où en sommes-nous ?

BiblioCours affiche son catalogue (chapitre 4), mais pour ajouter un cours il faut passer par le shell.

Dans ce chapitre, nous construisons deux outils de saisie complémentaires :

- des **formulaires publics**, par lesquels un éditeur crée et modifie des **brouillons** ;
- l'**administration Django**, par laquelle une personne habilitée relit et **publie** les cours.

Ce partage des rôles est une décision de conception : un éditeur qui rédige ne publie pas lui-même, et le formulaire ne propose donc pas le champ `statut`.

Pourquoi un formulaire HTML ne suffit pas

Un formulaire est une frontière entre le navigateur et l'application. Le navigateur peut aider avec `required`, `type="email"` ou `maxlength`, mais ces attributs ne sont pas des protections : n'importe quel client HTTP (un script, l'outil `curl`, un navigateur modifié) peut les contourner. La validation importante doit être exécutée côté serveur.

Django distingue trois étapes :

1. **liaison** : associer les données reçues à un formulaire ;
2. **validation** : vérifier les champs et les règles métier ;
3. **action** : sauvegarder ou effectuer une opération seulement si le formulaire est valide.

Un formulaire non valide doit être réaffiché avec les valeurs acceptables et les erreurs lisibles. Un formulaire valide doit éviter de créer deux fois la même ressource lorsque l'utilisateur actualise la page.

Ce dernier problème est résolu par le schéma **POST/Redirect/GET** (PRG) :

- | | | | |
|---------|-------------------------|--|-------|
| 1. GET | /cours/nouveau/ | → le serveur renvoie le formulaire vide | (200) |
| 2. POST | /cours/nouveau/ | → données invalides : formulaire réaffiché | (200) |
| 3. POST | /cours/nouveau/ | → données valides : sauvegarde, puis redirection | (302) |
| 4. GET | /cours/<slug>/modifier/ | → le navigateur suit la redirection | (200) |

Sans l'étape 3, l'utilisateur qui actualise la page après un succès renverrait le POST et créerait un doublon. Avec la redirection, l'actualisation répète seulement le GET de l'étape 4, qui est inoffensif.

Les deux états d'un formulaire

Un formulaire Django existe sous deux formes :

État	Création	is_valid()	Usage
non lié (<i>unbound</i>)	<code>CoursForm()</code>	toujours <code>False</code>	afficher un formulaire vide ou pré-rempli
lié (<i>bound</i>)	<code>CoursForm(request.POST)</code>	valide les données	traiter une soumission

Une fois `is_valid()` appelé, deux attributs deviennent disponibles : `form.cleaned_data` (les valeurs validées et converties en types Python) et `form.errors` (les messages d'erreur par champ).

Créer un ModelForm

Créez `cours/forms.py` :

```
from django import forms

from .models import Cours

class CoursForm(forms.ModelForm):
    class Meta:
        model = Cours
        fields = ["titre", "slug", "resume", "contenu", "matiere"]
        widgets = {
            "resume": forms.Textarea(attrs={"rows": 3}),
            "contenu": forms.Textarea(attrs={"rows": 12}),
        }

    def clean_titre(self):
        titre = self.cleaned_data["titre"].strip()
        if len(titre) < 5:
            raise forms.ValidationError(
                "Le titre doit comporter au moins cinq caractères."
            )
        return titre

    def clean(self):
        cleaned_data = super().clean()
        titre = cleaned_data.get("titre")
        resume = cleaned_data.get("resume")
        if titre and resume and titre.lower() in resume.lower():
            self.add_error(
                "resume",
                "Le résumé ne doit pas répéter le titre mot pour mot.",
            )
        return cleaned_data
```

`ModelForm` déduit les champs à partir du modèle et réutilise ses contraintes de base (longueur maximale, unicité, champ obligatoire). La liste `fields` doit être explicite : `fields = "__all__"` risque d'exposer un champ ajouté plus tard sans que l'on s'en rende compte (Django Software Foundation 2025i).

Notre liste omet volontairement `statut` et `date_publication`. Même si un utilisateur malveillant ajoutait `statut=publie` à la requête POST, Django **ignorerait** ce champ inconnu : un éditeur ne peut pas se publier lui-même. C'est une protection très concrète contre les modifications non autorisées de champs sensibles.

Les niveaux de validation

Quand on appelle `is_valid()`, Django enchaîne les validations dans cet ordre :

1. **par champ** : conversion et contraintes de base (obligatoire, longueur, format) ;
2. `clean_<champ>()` : règle propre à un champ (`clean_titre`) ;
3. `clean()` du formulaire : règle qui compare plusieurs champs ;
4. **validation du modèle** : `Model.clean()` (notre règle sur la date de publication) et contraintes d'unicité (`slug` unique).

Chaque méthode a un rôle précis :

- `clean_titre` valide un champ et se concentre sur sa valeur ; elle **doit renvoyer** la valeur nettoyée ;
- `clean` valide une relation entre plusieurs champs ; on y accède aux champs déjà validés via `cleaned_data.get(...)` (un champ invalide peut manquer) ;
- `Model.clean` porte une règle de modèle indépendante de ce formulaire.

Une validation ne doit pas seulement servir à afficher un message. Elle doit protéger l'invariant métier. Par exemple, si un cours publié doit toujours avoir un résumé, cette règle devrait être appliquée dans tous les chemins de création, pas uniquement sur la page HTML : on la place dans le modèle (voir l'exercice 7 du Section).

La vue de création

Ajoutez la route à `cours/urls.py`. Elle doit précéder la route générique `<slug:slug>/`, sans quoi nouveau serait traité comme le slug d'un cours. Voici la liste complète des routes à ce stade, qui inclut aussi celles de modification et de suppression :

```
urlpatterns = [
    path("", views.liste_cours, name="liste"),
    path("a-propos/", views.a_propos, name="a_propos"),
    path("nouveau/", views.creer_cours, name="creer"),
    path("matiere/<slug:slug>/", views.cours_par_matiere, name="par_matiere"),
    path("<slug:slug>/", views.detail_cours, name="detail"),
    path("<slug:slug>/modifier/", views.modifier_cours, name="modifier"),
    path("<slug:slug>/supprimer/", views.supprimer_cours, name="supprimer"),
]
```

Puis, dans `cours/views.py`, ajoutez les imports et la vue :

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect, render

from .forms import CoursForm
```

```
@login_required
def creer_cours(request):
    if request.method == "POST":
        form = CoursForm(request.POST)
        if form.is_valid():
            cours = form.save()
            messages.success(
                request, f"Le brouillon « {cours.titre} » a été créé."
            )
            return redirect("cours:modifier", slug=cours.slug)
        else:
            form = CoursForm()

    contexte = {"form": form, "titre_page": "Créer un cours"}
    return render(request, "cours/formulaire.html", contexte)
```

La méthode HTTP est une décision de sécurité et de sémantique :

- GET affiche le formulaire et ne doit pas modifier l'état ;
- POST transmet une intention de modification ;
- une redirection après succès empêche un rafraîchissement de répéter le POST.

Remarquez la structure `if POST ... else GET` : si le formulaire est **invalide**, on ne redirige pas, on laisse la vue atteindre le **render** final avec le formulaire lié, qui contient les valeurs saisies et les erreurs.

! Pourquoi rediriger vers la page de modification et non vers le détail ?

Un cours fraîchement créé est un **brouillon**, et `detail_cours` ne montre que les cours publiés : rediriger vers `cours.get_absolute_url()` mènerait à une 404. On redirige donc vers `cours:modifier`, qui existe quel que soit le statut. La destination d'une redirection doit toujours être une page que l'utilisateur peut effectivement voir.

Le décorateur `login_required` redirige les utilisateurs anonymes vers la page de connexion. Nous verrons la configuration de cette redirection, et son remplacement par un contrôle de permission, au Section .

i Tester sans page de connexion

La page de connexion n'existe pas encore (elle arrive au chapitre 6), donc un visiteur anonyme est redirigé vers `/accounts/login/`, qui renvoie une 404 : c'est attendu. Pour tester vos vues dès maintenant, connectez-vous par l'administration (`/admin/`, après `createsuperuser`, voir plus bas). La session ouverte est partagée par tout le site.

Le template du formulaire

`cours/templates/cours/formulaire.html` :

```
{% extends "cours/base.html" %}

{% block title %}{% titre_page %} - BiblioCours{% endblock %}

{% block content %}
    <h1>{% titre_page %}</h1>

    <form method="post" novalidate>
        {% csrf_token %}
```

```
{% if form.non_field_errors %}
    <div class="errors" role="alert">
        {{ form.non_field_errors }}
    </div>
{% endif %}

{% for field in form %}
    <div class="form-field">
        {{ field.label_tag }}
        {{ field }}
        {% if field.help_text %}
            <p class="help">{{ field.help_text }}</p>
        {% endif %}
        {% for error in field.errors %}
            <p class="error" role="alert">{{ error }}</p>
        {% endfor %}
    </div>
{% endfor %}

<button type="submit">Enregistrer</button>
<a href="{% url 'cours:liste' %}">Annuler</a>
{% if cours %}
    <a href="{% url 'cours:supprimer' cours.slug %}">Supprimer</a>
{% endif %}
</form>
{% endblock %}
```

Ce template sert à la création **et** à la modification : seul le titre de la page (`titre_page`) et la présence de `cours` dans le contexte changent. Le parcours `{% for field in form %}` donne un contrôle fin sur le rendu, mais on peut aussi écrire simplement `{ form.as_p }` pour un rendu rapide.

`{% csrf_token %}` est obligatoire pour un POST provenant d'un formulaire Django. Le jeton associe la requête à la session et protège contre une partie des attaques CSRF (Section). Il ne remplace ni l'authentification ni l'autorisation. Sans lui, le POST est refusé avec une erreur 403.

`novalidate` désactive la validation native du navigateur afin de rendre visibles les messages Django pendant l'apprentissage. Dans un projet réel, vous pouvez conserver la validation native en complément ; elle améliore l'expérience mais ne remplace jamais la validation serveur.

Modifier un objet existant

Le même `ModelForm` peut être lié à une instance :

```
@login_required
def modifier_cours(request, slug):
    cours = get_object_or_404(Cours, slug=slug)

    if request.method == "POST":
        form = CoursForm(request.POST, instance=cours)
        if form.is_valid():
            cours = form.save()
            messages.success(request, "Le cours a été mis à jour.")
            if cours.est_publie:
                return redirect(cours)
            return redirect("cours:modifier", slug=cours.slug)
    else:
```

```
form = CoursForm(instance=cours)

contexte = {"form": form, "cours": cours, "titre_page": "Modifier le cours"}
return render(request, "cours/formulaire.html", contexte)
```

La présence d'`instance` change le comportement : sans elle, `save()` crée un objet ; avec elle, `save()` met à jour l'objet. La vue utilise ici `get_object_or_404(Cours, slug=slug)` : à la différence de la page publique, elle accepte aussi les brouillons, puisqu'on veut pouvoir les éditer.

Après un succès, deux cas se présentent : un cours publié renvoie vers sa page publique (`redirect(cours)` utilise automatiquement `get_absolute_url()`), un brouillon revient sur le formulaire. La route de modification doit être distincte de la route de création et doit vérifier que l'utilisateur a le droit de modifier ce cours : ce contrôle d'autorisation sera ajouté au Section .

Supprimer : préférer une confirmation explicite

La suppression ne devrait pas être déclenchée par un lien GET :

```
<!-- À éviter : un robot ou un clic involontaire pourrait supprimer -->
<a href="{% url 'cours:supprimer' cours.slug %}">Supprimer</a>
```

Un lien GET est suivi par les robots d'indexation, préchargé par certains navigateurs et déclenché par un simple clic distrait. On affiche donc d'abord une **page de confirmation** (en GET, inoffensive), et c'est le **formulaire POST** de cette page, protégé par le jeton CSRF, qui supprime vraiment :

```
@login_required
def supprimer_cours(request, slug):
    cours = get_object_or_404(Cours, slug=slug)
    if request.method == "POST":
        titre = cours.titre
        cours.delete()
        messages.success(request, f"Le cours « {titre} » a été supprimé.")
        return redirect("cours:liste")
    return render(request, "cours/confirmer_suppression.html", {"cours": cours})
```

cours/templates/cours/confirmer_suppression.html :

```
{% extends "cours/base.html" %}

{% block title %}Supprimer - BiblioCours{% endblock %}

{% block content %}
<h1>Supprimer « {{ cours.titre }} » ?</h1>
<p>Cette action est définitive.</p>
<form method="post">
    {% csrf_token %}
    <button type="submit">Oui, supprimer</button>
    <a href="{% url 'cours:modifier' cours.slug %}">Annuler</a>
</form>
{% endblock %}
```

L'approbation visuelle n'est pas une mesure de sécurité ; l'autorisation et le POST protégé le sont. Une opération irréversible doit aussi être reflétée dans les tests et, selon le domaine, dans une stratégie d'archivage plutôt que de suppression physique.

L'administration Django

L'admin est une interface interne puissante pour les personnes qui gèrent le contenu. Elle ne remplace pas une interface publique : ses écrans, ses permissions et son modèle mental sont ceux d'un back-office.

Elle permet néanmoins de peupler BiblioCours rapidement et de **publier** les brouillons.

Dans `cours/admin.py` :

```
from django.contrib import admin, messages
from django.utils import timezone

from .models import Cours, Matiere

@admin.register(Matiere)
class MatiereAdmin(admin.ModelAdmin):
    list_display = ["nom", "slug"]
    search_fields = ["nom", "description"]
    prepopulated_fields = {"slug": ("nom",)}

@admin.register(Cours)
class CoursAdmin(admin.ModelAdmin):
    list_display = [
        "titre",
        "matiere",
        "statut",
        "date_creation",
        "date_publication",
    ]
    list_filter = ["statut", "matiere", "date_creation"]
    search_fields = ["titre", "resume", "contenu"]
    autocomplete_fields = ["matiere"]
    prepopulated_fields = {"slug": ("titre",)}
    readonly_fields = ["date_creation", "date_modification"]
    list_select_related = ["matiere"]
    actions = ["publier"]

@admin.action(description="Publier les cours sélectionnés")
def publier(self, request, queryset):
    maintenant = timezone.now()
    nombre = queryset.exclude(statut=Cours.Statut.PUBLIE).update(
        statut=Cours.Statut.PUBLIE,
        date_publication=maintenant,
        date_modification=maintenant,
    )
    self.message_user(request, f"{nombre} cours publié(s).", messages.SUCCESS)
```

Créez un compte administrateur :

```
uv run python manage.py createsuperuser
uv run python manage.py runserver
```

Visitez <http://127.0.0.1:8000/admin/>. Chaque option a un rôle précis :

Option	Effet
<code>list_display</code>	colonnes affichées dans la liste
<code>list_filter</code>	filtres latéraux
<code>search_fields</code>	champs interrogés par la barre de recherche
<code>autocomplete_fields</code>	champ de sélection avec recherche (exige <code>search_fields</code> sur l'admin de la cible : d'où <code>MatiereAdmin</code>)
<code>prepopulated_fields</code>	pré-remplit le slug depuis le titre pendant la saisie
<code>readonly_fields</code>	affiche un champ sans permettre de le modifier ; indispensable pour afficher les champs <code>auto_now</code>
<code>list_select_related</code>	précharge une relation pour éviter le N+1 dans la liste
<code>actions</code>	opérations en masse sur les lignes cochées

Essayez de créer un cours en choisissant le statut « Publié » **sans** date de publication : l'admin appelle `full_clean()`, donc la règle de `Cours.clean()` (chapitre 3) s'applique et affiche l'erreur à côté du champ. Les règles du modèle valent partout.

Une action personnalisée

L'action `publier` illustre deux conséquences du chapitre 3. D'abord, `update()` ne met **pas** à jour `auto_now` : nous renseignons donc `date_modification` nous-mêmes. Ensuite, `update()` ne lance pas `clean()` : l'action doit donc elle-même respecter la règle « publié date de publication », ce qu'elle fait en écrivant les deux champs ensemble. `exclude(statut=PUBLIE)` évite d'écraser la date d'un cours déjà publié.

L'admin et les performances

Si une liste affiche une relation, Django peut charger chaque matière séparément. `list_select_related = ["matiere"]` résout cela pour les colonnes de liste. Pour un contrôle total, on peut aussi redéfinir `get_queryset` :

```
def get_queryset(self, request):
    return super().get_queryset(request).select_related("matiere")
```

Mesurez avant d'optimiser, mais ne rendez pas l'admin inutilisable avec une requête N+1 lorsque le catalogue grandit.

Messages utilisateur

Les messages sont temporaires et associés à la prochaine réponse : on les « poste » dans une vue, puis ils s'affichent **une seule fois** sur la page suivante (typiquement après une redirection).

```
from django.contrib import messages

messages.success(request, "Le cours a été enregistré.")
messages.error(request, "Impossible de supprimer ce cours.")
```

Les niveaux disponibles sont `debug`, `info`, `success`, `warning` et `error`. Dans `base.html`, ajoutez le bloc d'affichage juste avant `{% block content %}` :

```
<main class="container">
  {% if messages %}
    <div class="messages" aria-live="polite">
      {% for message in messages %}
        <p class="message message-{{ message.tags }}">{{ message }}</p>
      {% endfor %}
    </div>
  {% endif %}

  {% block content %}{% endblock %}
</main>
```

Le contexte est disponible grâce à `django.contrib.messages.context_processors.messages`, présent dans la configuration initiale. La classe `message-{{ message.tags }}` (par exemple `message-success`) permet au CSS de colorer chaque niveau. Un message ne doit pas être la seule façon de communiquer une erreur : le formulaire doit aussi porter les erreurs au niveau du champ.

Checkpoint

Après `createsuperuser` et connexion via `/admin/` :

- ☐ `/cours/nouveau/` affiche un formulaire vide ;
- ☐ un titre de moins de cinq caractères réaffiche le formulaire avec l'erreur, **sans perdre** les autres valeurs ;
- ☐ une soumission valide crée un **brouillon** et vous redirige vers sa page de modification, avec un message vert ;
- ☐ actualiser la page (F5) après ce succès ne crée **pas** de doublon ;
- ☐ `/cours/<slug>/` du brouillon renvoie 404, même pour un utilisateur connecté ;
- ☐ l'action « Publier les cours sélectionnés » rend le cours visible sur `/cours/` ;
- ☐ `/cours/<slug>/supprimer/` affiche la confirmation ; seul le bouton POST supprime ;
- ☐ `uv run python manage.py check` ne signale aucun problème.

Erreurs fréquentes

Symptôme	Cause probable	Remède
Forbidden (403) CSRF verification failed formulaire invalide, mais aucune erreur affichée	{% csrf_token %} absent du formulaire le template ne parcourt pas <code>field.errors</code>	l'ajouter dans la balise <code><form method="post"></code> afficher les erreurs, ou utiliser <code>{ form.as_p }</code>
404 juste après la création d'un cours	redirection vers la page publique d'un brouillon	rediriger vers <code>cours:modifier</code>
ImproperlyConfigured: Creating a ModelForm without either the 'fields' attribute...	<code>Meta.fields</code> absent	lister explicitement les champs
admin.E039 ou erreur sur <code>autocomplete_fields</code> NoReverseMatch sur <code>cours:modifier</code>	<code>search_fields</code> absent de l'admin de la cible route absente ou mal nommée	ajouter <code>search_fields</code> à <code>MatiereAdmin</code> vérifier <code>cours/urls.py</code>
« Ce champ est obligatoire » sur un champ absent du formulaire anonyme redirigé vers une 404 <code>/accounts/login/</code>	champ du modèle ni obligatoire ni dans <code>fields</code> page de connexion pas encore créée	donner un <code>default</code> au modèle ou l'ajouter à <code>fields</code> normal : voir le Section

À retenir

- La validation importante se fait **côté serveur** ; les attributs HTML ne protègent rien.
- Le cycle est liaison → validation → action, suivi d'une redirection (PRG).
- **fields** est une liste explicite : un champ absent ne peut pas être modifié par une requête forgée.
- GET ne modifie jamais l'état ; suppression et modification passent par POST avec jeton CSRF.
- **instance=** fait passer un **ModelForm** de la création à la modification.
- Une règle métier se place dans le modèle pour valoir partout (formulaire, admin, scripts).
- **update()** contourne **clean()** et **auto_now** : on les gère soi-même.
- Les messages sont affichés une fois, après une redirection.

Exercices

1. Dans quel ordre Django exécute-t-il **clean_titre**, **clean** du formulaire et **Model.clean** ? Donnez un exemple de règle adaptée à chaque niveau.
2. Que se passe-t-il, pas à pas, si un utilisateur actualise la page juste après une création réussie, selon que la vue redirige ou non ?
3. Pourquoi la suppression passe-t-elle par une page de confirmation et un POST plutôt que par un lien direct ?
4. Pourquoi **statut** n'est-il pas dans **CoursForm.Meta.fields** ? Que se passerait-il si **fields = "__all__"** était utilisé ?
5. Quelle est la différence entre **null=True** et **blank=True** du point de vue d'un formulaire ? Que se passe-t-il pour **date_publication** ?
6. Ajoutez un **ModelForm** pour **Matiere** avec une validation de nom unique insensible à la casse (« Python » et « python » sont des doublons). Où placez-vous la règle ?
7. Ajoutez à l'administration une action **Repasser en brouillon**. Que devient **date_publication** ? Justifiez votre choix.
8. Ajoutez des libellés (**labels**) et des textes d'aide (**help_texts**) en français dans **CoursForm.Meta**. Vérifiez qu'ils s'affichent.
9. Effectuez manuellement les trois scénarios de **creer_cours** (GET, POST valide, POST invalide) avec l'onglet *Réseau* des outils de développement du navigateur. Notez les méthodes, statuts et redirections. Nous les automatiserons au Section .
10. Dans l'admin, essayez d'enregistrer un cours « Publié » sans date de publication, puis avec une date. Quel message obtenez-vous ? D'où vient-il ?
11. Rendez le champ **slug** facultatif dans le formulaire et générez-le automatiquement à partir du titre avec **django.utils.text.slugify** s'il est vide. Attention aux doublons : que faites-vous si le slug existe déjà ?
12. Ajoutez dans **clean_slug** une liste de slugs **réservés** (nouveau, a-propos, matiere) qui sont refusés. Expliquez le lien avec l'ordre des routes du Section .
13. Empêchez la modification du slug d'un cours existant (changer le slug casse les liens). Indice : **self.fields["slug"].disabled = True** dans **__init__** quand **self.instance.pk** existe.

Authentification, sécurité et fichiers statiques

Objectifs d'apprentissage

À la fin de ce chapitre, vous saurez :

- distinguer authentification et autorisation, et expliquer le rôle d'une session ;
- ajouter la connexion et la déconnexion, et protéger des vues avec `login_required` et `permission_required` ;
- créer un groupe de permissions (« Éditeurs ») et adapter les templates aux droits de l'utilisateur ;
- expliquer et neutraliser les risques CSRF, XSS, injection SQL et redirection ouverte ;
- sortir les secrets du code grâce aux variables d'environnement ;
- configurer correctement fichiers statiques et fichiers téléversés ;
- appliquer une checklist de sécurité avant publication.

Où en sommes-nous ?

Les vues de création, de modification et de suppression existent (chapitre 5), mais elles sont protégées uniquement par `login_required`, et le site n'a pas de page de connexion : il faut passer par l'administration. Surtout, **tout utilisateur connecté peut tout faire**. Ce chapitre installe un vrai contrôle d'accès, puis range la configuration sensible là où elle doit être.

Authentifier n'est pas autoriser

Deux questions différentes sont souvent confondues :

- **authentification** : « qui est cette personne ? » ;
- **autorisation** : « a-t-elle le droit de faire cette action ? ».

Un utilisateur connecté peut avoir le droit de lire un cours mais pas de le supprimer. La présence de `request.user` ne suffit donc jamais à autoriser une opération.

Django fournit un système d'utilisateurs, de sessions, de groupes et de permissions. Il faut le configurer et l'utiliser de manière explicite plutôt que d'inventer un mécanisme de mot de passe. Les recommandations de sécurité du framework et les contrôles OWASP complètent cette compréhension ([Django Software Foundation 2025f](#); [OWASP Foundation 2021](#)).

Comment fonctionne une session

HTTP est « sans mémoire » : chaque requête est indépendante, et le serveur ne sait pas, de lui-même, que deux requêtes viennent de la même personne. Django résout cela avec une **session** :

1. `POST /connexion/` (identifiant + mot de passe)
2. Django vérifie le mot de passe, crée une session (une ligne dans la table `django_session`) et répond : `Set-Cookie: sessionid=abc123...`
3. `GET /cours/nouveau/` + `Cookie: sessionid=abc123...` (automatique, à chaque requête)
4. `SessionMiddleware` retrouve la session ; `AuthenticationMiddleware` place l'utilisateur dans `request.user`

Le navigateur ne conserve donc qu'un **identifiant de session** dans un cookie ; les données restent côté serveur. Les mots de passe ne sont jamais stockés en clair : Django ne conserve qu'une empreinte (un *hash* salé), d'où l'importance de créer les comptes avec `create_user` ou l'administration, et non en écrivant dans la base à la main.

Ajouter les routes d'authentification

Dans `config/urls.py` :

```
from django.contrib import admin
from django.contrib.auth import views as auth_views
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("cours/", include("cours.urls")),
    path(
        "connexion/",
        auth_views.LoginView.as_view(template_name="registration/login.html"),
        name="login",
    ),
    path("deconnexion/", auth_views.LogoutView.as_view(), name="logout"),
]
```

LoginView et LogoutView sont des vues **fournies par Django** : elles traitent le formulaire, vérifient le mot de passe et gèrent la session. Nous n'avons qu'à fournir le template. Ajoutez dans `config/settings.py` :

```
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "cours:liste"
LOGOUT_REDIRECT_URL = "cours:liste"
```

LOGIN_URL est la page vers laquelle `login_required` redirige les anonymes ; LOGIN_REDIRECT_URL la destination après une connexion réussie (sauf si un paramètre `next` existe) ; LOGOUT_REDIRECT_URL la destination après la déconnexion. Ces trois valeurs peuvent être des noms de routes.

Le template de connexion doit se trouver à l'emplacement attendu :

```
templates/
  registration/
    login.html
```

Ce dossier `templates/` est à la **racine du projet** et non dans l'app : c'est un template qui concerne le site entier. Pour qu'il soit trouvé, déclarez-le dans le réglage `TEMPLATES` existant :

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        # ... le reste inchangé
    },
]
```

DIRS liste les dossiers « globaux » où chercher des templates ; APP_DIRS les dossiers `templates/` de chaque app. `templates/registration/login.html` :

```
{% extends "cours/base.html" %}

{% block title %}Connexion - BiblioCours{% endblock %}

{% block content %}
<h1>Se connecter</h1>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Se connecter</button>
</form>
{% endblock %}
```

Ce formulaire n'a pas d'attribut `action` : il est donc envoyé à l'URL courante, **paramètres compris**.

Quand un anonyme est redirigé vers `/connexion/?next=/cours/nouveau/`, ce paramètre `next` accompagne le POST et `LoginView` renvoie l'utilisateur vers la page qu'il voulait atteindre. Django vérifie que `next` pointe vers un hôte autorisé (voir « Redirections » plus bas).

Dans `base.html`, adaptez la navigation (remplacez le bloc `<nav>`) :

```
<nav aria-label="Navigation principale">
  <a href="{% url 'cours:liste' %}">BiblioCours</a>
  <a href="{% url 'cours:a_propos' %}">À propos</a>
  {% if perms.cours.add_cours %}
    <a href="{% url 'cours:creer' %}">Nouveau cours</a>
  {% endif %}
  {% if user.is_authenticated %}
    <span>Bonjour {{ user.get_username }}</span>
    <form method="post" action="{% url 'logout' %}" class="inline-form">
      {% csrf_token %}
      <button type="submit">Se déconnecter</button>
    </form>
  {% else %}
    <a href="{% url 'login' %}?next={{ request.path|urlencode }}">
      Se connecter
    </a>
  {% endif %}
</nav>
```

Les variables `user` et `perms` viennent du *context processor*

`django.contrib.auth.context_processors.auth` : elles sont disponibles dans tous les templates. Sur les versions récentes de Django, la déconnexion est traitée par POST. Utiliser un formulaire respecte le fait que la déconnexion modifie l'état de la session ; un simple lien déclencherait une erreur 405 `Method Not Allowed`.

Protéger une vue et une action

Le décorateur protège la présence d'une session :

```
from django.contrib.auth.decorators import login_required

@login_required
def creer_cours(request):
    ...
```

Pour une permission :

```
from django.contrib.auth.decorators import login_required, permission_required

@login_required
@permission_required("cours.add_cours", raise_exception=True)
def creer_cours(request):
    ...
```

Chaque modèle reçoit automatiquement quatre permissions : **add**, **change**, **delete** et **view**. Leur nom suit le modèle `app.action_model`, par exemple `cours.add_cours`.

Remarquez l'**empilement** des décorateurs, dans cet ordre. Il fournit trois comportements distincts :

Qui appelle la vue ?	Résultat
un visiteur anonyme	redirection vers la connexion (<code>login_required</code>)
un utilisateur connecté sans la permission	403 Forbidden (<code>raise_exception=True</code>)
un utilisateur connecté avec la permission	la vue s'exécute

`raise_exception=True` renvoie une réponse interdite (403) au lieu de rediriger silencieusement. Sans `login_required` au-dessus, un anonyme recevrait aussi un 403 plutôt qu'une invitation à se connecter. Choisissez le comportement en fonction du contexte : une app publique peut vouloir masquer l'existence d'une ressource ; une interface interne doit souvent expliquer le manque de permission.

Voici la protection finale des trois vues du chapitre 5 (seul l'en-tête change, le corps reste identique) :

```
@login_required
@permission_required("cours.add_cours", raise_exception=True)
def creer_cours(request): ...

@login_required
@permission_required("cours.change_cours", raise_exception=True)
def modifier_cours(request, slug): ...

@login_required
@permission_required("cours.delete_cours", raise_exception=True)
def supprimer_cours(request, slug): ...
```

Un groupe « Éditeurs »

Plutôt que d'attribuer des permissions utilisateur par utilisateur, on les regroupe. Dans l'administration :

1. **Groupes** → **Ajouter** : nommez le groupe Éditeurs ;
2. choisissez les permissions `cours | cours | Can add cours, Can change cours et Can view cours` (pas `Can delete cours`) ;
3. **Utilisateurs** → **Ajouter** : créez un compte « edith » avec un mot de passe, puis, dans sa fiche, ajoutez-la au groupe Éditeurs.

Ce compte n'a pas besoin de la case « Statut équipe » (`is_staff`) pour utiliser le site : cette case ne sert qu'à accéder à l'administration. Les permissions **add** et **change** suffisent pour créer et modifier des cours via le site ; seule la suppression leur est refusée. Seuls les comptes `is_staff` ayant la permission `change_cours` peuvent publier dans l'admin.

💡 Masquer n'est pas protéger

Le lien « Nouveau cours » n'apparaît que pour qui possède la permission (`{% if perms.cours.add_cours %}`) : c'est de l'**ergonomie**. La **sécurité**, elle, est dans le décorateur de la vue : un utilisateur qui connaît l'URL `/cours/nouveau/` mais n'a pas la permission reçoit un 403. Faites toujours les deux, et ne comptez jamais sur le masquage seul.

Dans `detail.html`, vous pouvez de la même façon n'afficher un lien « Modifier » qu'aux personnes autorisées :

```
{% if perms.cours.change_cours %}
<p><a href="{% url 'cours:modifier' cours.slug %}">Modifier ce cours</a></p>
{% endif %}
```

i Limite : permissions globales et par objet

Les permissions de Django portent sur un **modèle entier** : qui possède `change_cours` peut modifier **tous** les cours. Autoriser seulement l'auteur d'un cours à le modifier demande une règle supplémentaire par objet (par exemple comparer `cours.auteur` à `request.user`), ce que l'exercice 12 explore.

CSRF : protéger les requêtes qui changent l'état

Une attaque CSRF (*Cross-Site Request Forgery*) amène le navigateur d'une victime déjà connectée à envoyer une requête vers un site où elle est authentifiée. Exemple : une page piégée contient un formulaire invisible qui envoie un POST vers `/cours/mon-cours/supprimer/`. Le navigateur peut joindre les cookies automatiquement, donc la requête arrive avec la session de la victime ; le serveur doit donc exiger un jeton que le site attaquant ne peut pas lire.

Dans un formulaire interne :

```
<form method="post">
  {% csrf_token %}
  ...
</form>
```

`CsrfViewMiddleware` compare le jeton du formulaire à celui du cookie et refuse le POST (403) en cas de différence. Pour un appel JavaScript, le jeton doit être envoyé dans l'en-tête `X-CSRFToken`, selon la configuration Django. Ne désactivez pas `CsrfViewMiddleware` (ni `@csrf_exempt`) pour contourner une erreur : cherchez plutôt pourquoi le cookie ou le jeton ne correspond pas ([Django Software Foundation 2025a](#)).

Échappement, injection et entrées utilisateur

Une règle domine ce chapitre : **toute donnée venant de l'extérieur est hostile jusqu'à preuve du contraire** — paramètres d'URL, formulaires, en-têtes, fichiers, et même contenu saisi en administration.

XSS et templates

Une attaque XSS (*Cross-Site Scripting*) injecte du JavaScript dans une page vue par d'autres utilisateurs. Django échappe les variables dans les templates par défaut :

```
{{ commentaire.texte }}
```

Si un utilisateur enregistre `<img src=x onerror=alert(1)>`, cette valeur doit rester du texte : Django envoie `<img src=x ...>`, que le navigateur affiche tel quel au lieu de l'exécuter. Le filtre **safe** supprime la protection :

```
{{ contenu_html|safe }}
```

Ne l'utilisez que si le contenu a été nettoyé par une bibliothèque adaptée à votre format et à votre modèle de menace. Le fait qu'une donnée vienne de l'admin ne garantit pas que tous les administrateurs soient dignes de confiance ni que le champ soit toujours contrôlé.

Injection SQL

Une injection SQL glisse du SQL dans une requête construite par concaténation : avec `f"... WHERE titre = '{terme}'"`, un terme comme `' OR 1=1 --` change le sens de la requête. L'ORM paramètre les valeurs, c'est-à-dire qu'il les transmet séparément de la requête :

```
Cours.objects.filter(titre__icontains=terme)
```

Évitez de construire du SQL avec une f-string. Si une requête SQL brute est nécessaire, utilisez des paramètres et expliquez la raison dans le code :

```
Cours.objects.raw(
    "SELECT * FROM cours_cours WHERE titre LIKE %s",
    [f"%{terme}%"],
)
```

Le nom de la table, `cours_cours`, suit la règle vue au chapitre 3. Le SQL brut n'est pas « plus professionnel ». Il augmente la responsabilité du développeur concernant les paramètres, les noms de tables, les performances et la portabilité.

Redirections

Une redirection ouverte permet à un attaquant de fabriquer un lien légitime (`votre-site.fr/...?next=https://site-pirate.example`) qui renvoie vers un site malveillant. Ne redirigez pas aveuglément vers une URL fournie par l'utilisateur. Un paramètre `next` doit être validé comme une destination sûre :

```
from django.shortcuts import redirect
from django.utils.http import url_has_allowed_host_and_scheme

def apres_action(request):
    suivant = request.POST.get("next", "")
    if url_has_allowed_host_and_scheme(
        suivant,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        return redirect(suivant)
    return redirect("cours:liste")
```

`LoginView` applique déjà ce contrôle à son paramètre `next`.

Ce que Django fait pour vous, et ce qui reste à votre charge

Risque	Protection fournie	Votre responsabilité
XSS	échappement automatique des templates	ne pas abuser de <code> safe</code>
injection SQL	paramétrage par l'ORM	ne pas concaténer de SQL
CSRF	<code>CsrfViewMiddleware</code> et <code>{% csrf_token %}</code>	mettre le jeton dans tout formulaire POST
mots de passe	stockage haché, validateurs	ne jamais les stocker ni les journaliser
autorisation	permissions, décorateurs	les appliquer sur chaque vue sensible
secrets	—	ne pas les mettre dans le dépôt

Secrets et configuration

Jusqu'ici, `SECRET_KEY`, `DEBUG` et `ALLOWED_HOSTS` sont écrits en dur dans `settings.py`, donc dans Git.

C'est dangereux : toute personne qui lit le dépôt connaît la clé qui signe vos sessions. La règle est de **séparer la configuration du code** : le code est identique partout, et chaque environnement (poste de développement, serveur) fournit ses propres valeurs par des **variables d'environnement**.

Modifiez le début de `config/settings.py` :

```
import os
from pathlib import Path

from django.core.exceptions import ImproperlyConfigured

BASE_DIR = Path(__file__).resolve().parent.parent

def env_requis(nom):
    """Lit une variable d'environnement obligatoire."""
    valeur = os.environ.get(nom)
    if not valeur:
        raise ImproperlyConfigured(
            f"La variable d'environnement {nom} est requise "
            "(voir le fichier .env.example)."
        )
    return valeur

def env_liste(nom):
    """Lit une liste de valeurs séparées par des virgules."""
    brut = os.environ.get(nom, "")
    return [valeur.strip() for valeur in brut.split(",") if valeur.strip()]

SECRET_KEY = env_requis("DJANGO_SECRET_KEY")
DEBUG = os.environ.get("DJANGO_DEBUG", "false").lower() == "true"
ALLOWED_HOSTS = env_liste("DJANGO_ALLOWED_HOSTS")
```

Deux choix méritent d'être expliqués :

- `DEBUG` vaut **False par défaut** : si la variable est oubliée en production, le site reste sûr (on dit qu'il « échoue en sécurité »). Le mode debug doit être demandé explicitement ;

- l'absence de `DJANGO_SECRET_KEY` provoque une erreur **lisible** (`ImproperlyConfigured`) plutôt qu'un `KeyError` obscur, et empêche de démarrer avec une clé par défaut.

Fournir les valeurs en local

Créez à la racine un fichier `.env` (déjà ignoré par Git depuis le chapitre 1) :

```
DJANGO_SECRET_KEY=une-cle-locale-longue-et-aleatoire
DJANGO_DEBUG=true
DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost
```

Pour fabriquer une clé aléatoire :

```
uv run python -c "import secrets; print(secrets.token_urlsafe(50))"
```

Créez aussi un fichier `.env.example` **sans secret**, qui, lui, est versionné : il documente les variables attendues.

```
DJANGO_SECRET_KEY=remplacez-moi
DJANGO_DEBUG=true
DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost
```

! Changement de routine : charger `.env`

`uv run` peut charger le fichier `.env` pour vous. La méthode la plus simple est de le préciser pour chaque commande :

```
uv run --env-file .env python manage.py runserver
uv run --env-file .env python manage.py test
```

Pour éviter de répéter l'option, vous pouvez définir `UV_ENV_FILE` pour la session du terminal. Sous macOS/Linux :

```
export UV_ENV_FILE=.env
```

Sous PowerShell :

```
$env:UV_ENV_FILE = ".env"
```

Dans ce terminal, les commandes `uv run python manage.py runserver`, `test` et `check` chargeront le même fichier. Si une nouvelle fenêtre de terminal est ouverte, il faut y redéfinir la variable, ou utiliser `--env-file .env`. Si Django signale que `DJANGO_SECRET_KEY` manque, vérifiez que `.env` se trouve à la racine du projet et que la commande charge bien ce fichier. Ne remplacez pas cette erreur par une clé secrète par défaut.

Ne commitez ni clé secrète, ni mot de passe de base, ni token de service. Un fichier `.env` peut aider en local mais doit être ignoré par Git et ne doit pas être copié dans une archive de déploiement. En production, utilisez le système de secrets de l'hébergeur (voir Section).

Fichiers statiques

Les fichiers statiques sont le CSS, le JavaScript et les images livrés avec le code : ils ne changent pas d'une requête à l'autre. Django les trouve grâce à des **finders** (chercheurs) qui explorent deux types d'emplacements :

Emplacement	Réglage	Exemple
dossier <code>static/</code> d'une app	automatique (<code>AppDirectoriesFinder</code>)	<code>cours/static/cours/site.css</code>
dossiers du projet	<code>STATICFILES_DIRS</code>	<code>static/</code> à la racine

Notre feuille `cours/static/cours/site.css` (créée au chapitre 4) est donc déjà trouvée sans réglage. Configurez la partie nécessaire au déploiement dans `config/settings.py` :

```
STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Et, si vous voulez des fichiers statiques propres au projet (hors app), ajoutez :

```
STATICFILES_DIRS = [BASE_DIR / "static"]
```

(le dossier `static/` doit alors exister, sinon `check` émet un avertissement).

Le template les charge avec `{% load static %}` et `{% static 'cours/site.css' %}`. En développement, `runserver` les sert si `DEBUG=True`. En production, la commande suivante rassemble les fichiers de **tous** les emplacements :

```
uv run python manage.py collectstatic
```

`STATIC_ROOT` est un dossier de sortie ; il ne doit pas être confondu avec `STATICFILES_DIRS`, qui contient les sources. Un serveur web ou un stockage objet servira ensuite le contenu rassemblé. Pour diagnostiquer, `findstatic` indique où Django trouve un fichier :

```
uv run python manage.py findstatic cours/site.css
```

Téléversements et fichiers média

Un fichier envoyé par un utilisateur n'est pas un fichier statique : il n'existe pas au moment du déploiement et n'est pas de confiance. Configurez :

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

`MEDIA_ROOT` est le dossier où Django écrit les fichiers envoyés ; `MEDIA_URL` le préfixe sous lequel on les sert. Voici l'idée, **à titre d'illustration** (nous ne modifions pas le modèle `Cours` pour l'instant : voir l'exercice 11) :

```
document = models.FileField(upload_to="cours/%Y/%m", blank=True)
```

`upload_to` indique le sous-dossier de destination, ici organisé par année et mois. Un formulaire qui accepte des fichiers doit déclarer `enctype="multipart/form-data"` et la vue lire `request.FILES` en plus de `request.POST`. En développement seulement, on peut servir les médias depuis `config/urls.py` :

```
from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Un upload doit être limité par taille et type, renommé par Django et stocké avec des permissions correctes. Ne vous fiez ni au nom du fichier ni au type déclaré par le navigateur : ils sont fournis par l'utilisateur. Ne servez pas aveuglément un fichier comme du HTML actif. Pour des données sensibles ou volumineuses, un stockage séparé avec des URLs temporaires est souvent plus adapté.

Checklist de sécurité locale

Avant de publier :

- `DEBUG=False`;
- `SECRET_KEY` fourni par l'environnement ;
- `ALLOWED_HOSTS` renseigné ;
- HTTPS activé et cookies sécurisés configurés ;
- protection CSRF conservée ;
- permissions testées avec un utilisateur normal ;
- pas de secret dans Git ;
- uploads limités et traités comme non fiables ;
- `python manage.py check --deploy` exécuté.

La sécurité n'est pas une étape à la fin. Chaque champ, route et action doit répondre à la question : **qui peut faire quoi, avec quelles données, et que se passe-t-il si la requête est fabriquée à la main ?**

Checkpoint

- ☐ `/connexion/` affiche le formulaire ; connectez-vous avec « edith » ;
- ☐ « edith » voit « Nouveau cours » et peut créer un brouillon ;
- ☐ « edith » reçoit un **403** sur `/cours/<slug>/supprimer/` ;
- ☐ un visiteur anonyme visitant `/cours/nouveau/` est redirigé vers `/connexion/?next=/cours/nouveau/` ;
- ☐ un utilisateur connecté sans aucune permission reçoit un 403 sur `/cours/nouveau/` ;
- ☐ la déconnexion fonctionne (formulaire POST) ;
- ☐ sans `.env` chargé, le démarrage affiche l'erreur `ImproperlyConfigured` ; avec, le site démarre ;
- ☐ `git status` ne montre pas `.env` mais montre `.env.example`.

Erreurs fréquentes

Symptôme	Cause probable	Remède
403 CSRF verification failed	jeton absent, ou cookie bloqué/périmé	<code>{% csrf_token %}</code> dans le formulaire ; ne pas désactiver le middleware
anonyme redirigé vers une 404 /accounts/login/	LOGIN_URL non défini	ajouter <code>LOGIN_URL = "login"</code>
TemplateDoesNotExist: registration/login.html	DIRS non configuré ou dossier mal placé	<code>DIRS = [BASE_DIR / "templates"]</code> et chemin <code>templates/registration/login.html</code>

Symptôme	Cause probable	Remède
NoReverseMatch: 'login'	route de connexion absente ou non nommée	vérifier <code>name="login"</code> dans <code>config/urls.py</code>
405 Method Not Allowed à la déconnexion	déconnexion via un lien GET	utiliser un formulaire POST
ImproperlyConfigured: ... DJANGO_SECRET_KEY	.env non chargé	UV_ENV_FILE=.env ou uv run --env-file .env
DisallowedHost	hôte absent de ALLOWED_HOSTS	l'ajouter à DJANGO_ALLOWED_HOSTS
CSS introuvable (404 sur /static/...)	mauvais emplacement, ou DEBUG=False en local	findstatic; vérifier DJANGO_DEBUG=true
la page de connexion est affichée sans retour vers la page voulue	next perdu	ne pas mettre d'action sur le formulaire de connexion

À retenir

- **Authentifier** = qui ? **Autoriser** = a-t-il le droit ? Les deux sont nécessaires et distincts.
- Une session est un cookie contenant un identifiant ; les données sont côté serveur.
- `login_required` + `permission_required(raise_exception=True)` donnent : redirection pour l'anonyme, 403 pour l'utilisateur sans droit.
- Masquer un lien est de l'ergonomie ; le contrôle dans la vue est la sécurité.
- Toute entrée est hostile : échappement des templates, ORM paramétré, redirections validées, jeton CSRF.
- Les secrets viennent de l'environnement, jamais du dépôt ; `DEBUG` est faux par défaut.
- Fichiers statiques (code, `collectstatic`) et média (utilisateurs, non fiables) sont deux choses différentes.

Exercices

1. Classez chacune de ces questions en « authentification » ou « autorisation » : (a) le mot de passe est-il correct ? (b) cette personne peut-elle supprimer un cours ? (c) le cookie de session est-il valide ?

(d) cet utilisateur appartient-il au groupe « Éditeurs » ?
2. Expliquez en trois phrases ce qu'est une attaque CSRF et comment le jeton l'empêche.
3. Pourquoi empile-t-on `@login_required` au-dessus de `@permission_required(..., raise_exception=True)` ? Que se passerait-il sans `login_required` pour un anonyme ?
4. Donnez un exemple concret où `|safe` est dangereux. Que se passe-t-il pour `{{ "<b>Salut</b>" }}` et pour `{{ "<b>Salut</b>"|safe }}` ?
5. Que reproche-t-on à chacun de ces extraits ? (a) `Cours.objects.raw(f"SELECT * FROM cours_cours WHERE titre = '{terme}')` ;

(b) `return redirect(request.GET["next"])` ; (c) `@csrf_exempt` sur la vue de création.
6. Créez le groupe **Éditeurs** avec uniquement les permissions de création, de modification et de consultation. Connectez-vous avec un compte membre : que voyez-vous ? Que se passe-t-il à l'adresse de suppression ?
7. Ajoutez une page `/cours/brouillons/` qui liste les brouillons, visible uniquement des utilisateurs ayant la permission `cours.change_cours`. Pensez à l'ordre des routes et au lien dans la navigation.
8. Testez un POST sans jeton CSRF et observez la réponse ; ne désactivez pas la protection.

Indice : supprimez temporairement `{% csrf_token %}` d'un formulaire, soumettez-le, puis remettez-le. Une alternative en test automatique sera vue au Section : le client de test ignore le CSRF par défaut, sauf si on le demande.

9. Créez dans l'admin un cours dont le titre est `<script>alert(1)</script>` et affichez-le sur la liste. Que voyez-vous ? Quel mécanisme vous protège ?
10. Utilisez `findstatic` pour localiser `cours/site.css`. Créez ensuite un dossier `static/` à la racine, déclarez-le dans `STATICFILES_DIRS`, ajoutez-y un petit fichier CSS supplémentaire, chargez-le dans `base.html`, puis lancez `collectstatic` et observez le contenu de `staticfiles/`.
11. **Téléversement.** Ajoutez un champ `document = FileField(blank=True)` à `Cours`, créez la migration, ajoutez-le au formulaire (n'oubliez pas `enctype="multipart/form-data"` et `request.FILES`). Ajoutez ensuite une validation de taille et d'extension.
12. **Autorisation par objet.** Ajoutez un champ `auteur` (clé étrangère vers l'utilisateur, `null=True`, `on_delete=SET_NULL`), renseignez-le à la création, puis n'autorisez la modification qu'à l'auteur ou à un superutilisateur. Quel statut HTTP renvoyez-vous aux autres ? Pourquoi une migration avec `null=True` plutôt qu'obligatoire ?
13. Lancez `uv run python manage.py check --deploy` avec votre configuration locale. Notez chaque avertissement ; lesquels disparaissent avec `DEBUG=False` ? Gardez cette liste pour la Section .
14. Personnalisez la page d'erreur 403 avec un template `403.html` clair pour les utilisateurs.

Tester, diagnostiquer et faire évoluer le code

Un test décrit un contrat

Un test utile ne vérifie pas que l'implémentation ressemble à ce que le développeur avait en tête. Il vérifie un comportement observable :

lorsqu'un visiteur demande la liste, la réponse est 200 et ne montre que les cours publiés.

Cette formulation résiste mieux aux refactorings que « la vue appelle telle fonction ». Les tests de Django peuvent utiliser une base de données de test, un client HTTP de test et les outils `reverse` et `assertContains` ([Django Software Foundation 2025g](#)). Cette manière de partir du comportement attendu rejoint l'approche de développement piloté par les tests décrite par Percival et Gregory ([Percival et Gregory 2024](#)).

Lancer la suite

Django fournit une commande intégrée :

```
uv run python manage.py test
```

Elle découvre les classes `SimpleTestCase`, `TestCase` et les fonctions de test dans les modules dont le nom commence généralement par `test`. La base de test est créée temporairement et détruite à la fin.

Ajoutez un dossier dédié :

```
cours/  
  tests/  
    __init__.py  
    test_models.py  
    test_views.py  
    test_forms.py
```

Tester le modèle

`cours/tests/test_models.py` :

```
from django.test import TestCase  
  
from cours.models import Cours, Matiere  
  
class CoursModelTests(TestCase):  
    @classmethod  
    def setUpTestData(cls):  
        cls.matiere = Matiere.objects.create(  
            nom="Informatique",
```

```
        slug="informatique",
    )

    def test_str_retourne_le_titre(self):
        cours = Cours(
            titre="Python",
            slug="python",
            resume="Un résumé suffisamment long.",
            contenu="Du contenu.",
            matiere=self.matiere,
        )
        self.assertEqual(str(cours), "Python")

    def test_url_absolue_utilise_le_slug(self):
        cours = Cours.objects.create(
            titre="Python",
            slug="python",
            resume="Un résumé suffisamment long.",
            contenu="Du contenu.",
            matiere=self.matiere,
        )
        self.assertEqual(cours.get_absolute_url(), "/cours/python/")
```

`setUpTestData` crée les données une fois pour la classe, ce qui est souvent plus rapide que `setUp`, exécuté avant chaque méthode. Utilisez `setUp` si le test doit modifier son jeu de données sans contaminer les autres tests.

Le test ne suppose pas que la table `cours_cours` existait déjà : Django applique les migrations à la base isolée. C'est une raison de tester les migrations sur une base vide plutôt que de compter sur votre SQLite locale.

Tester les URLs et les vues

`cours/tests/test_views.py` :

```
from django.test import TestCase
from django.urls import reverse

from cours.models import Cours, Matiere

class ListeCoursTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        matiere = Matiere.objects.create(
            nom="Informatique",
            slug="informatique",
        )
        Cours.objects.create(
            titre="Cours visible",
            slug="visible",
            resume="Résumé visible.",
            contenu="Contenu visible.",
            matiere=matiere,
            statut=Cours.Statut.PUBLIE,
        )
```

```

Cours.objects.create(
    titre="Cours caché",
    slug="cache",
    resume="Résumé interne.",
    contenu="Brouillon.",
    matiere=matiere,
    statut=Cours.Statut.BROUILLON,
)

def test_liste_repond_200(self):
    response = self.client.get(reverse("cours:liste"))

    self.assertEqual(response.status_code, 200)
    self.assertTemplateUsed(response, "cours/liste.html")
    self.assertContains(response, "Cours visible")
    self.assertNotContains(response, "Cours caché")

def test_detail_inconnu_renvoye_404(self):
    response = self.client.get(
        reverse("cours:detail", kwargs={"slug": "inexistant"})
    )

    self.assertEqual(response.status_code, 404)

```

`self.client` simule un navigateur sans lancer de serveur. `reverse` fait apparaître les erreurs de nom de route immédiatement. `assertNotContains` est important ici : un filtre qui afficherait aussi les brouillons serait une régression fonctionnelle et potentiellement une fuite d'information.

Tester l'authentification et les permissions

```

from django.contrib.auth import get_user_model

User = get_user_model()

class CreationCoursTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username="alice",
            password="mot-de-passe-de-test",
        )

    def test_visiteur_redirige_vers_connexion(self):
        response = self.client.get(reverse("cours:creer"))

        self.assertEqual(response.status_code, 302)
        self.assertIn("/connexion/", response.url)

    def test_utilisateur_connecte_peut_afficher_le_formulaire(self):
        self.client.force_login(self.user)

        response = self.client.get(reverse("cours:creer"))

        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Créer un cours")

```

`force_login` évite de tester le formulaire de connexion en même temps que la vue de création. Un test doit isoler le comportement qu'il décrit.

Tester un formulaire

```
from django.test import TestCase

from cours.forms import CoursForm
from cours.models import Matiere

class CoursFormTests(TestCase):
    def setUp(self):
        self.matiere = Matiere.objects.create(
            nom="Python",
            slug="python",
        )

    def test_titre_trop_court_invalide_le_formulaire(self):
        form = CoursForm(
            data={
                "titre": "Py",
                "slug": "py",
                "resume": "Un résumé.",
                "contenu": "Contenu.",
                "matiere": self.matiere.pk,
            }
        )

        self.assertFalse(form.is_valid())
        self.assertIn("titre", form.errors)
```

Le test appelle `is_valid()` avant de consulter `errors`. Les valeurs de `ForeignKey` transmises à un formulaire HTTP sont des identifiants, pas des instances Python.

Déboguer sans deviner

Quand un test échoue, lisez le premier échec et son traceback. Puis répondez aux questions suivantes :

1. l'échec est-il dans l'arrangement des données, l'action, ou l'assertion ?
2. l'exception appartient-elle au code de production ou au test ?
3. le comportement attendu a-t-il changé volontairement ?
4. un autre test échoue-t-il à cause d'un état partagé ?

Les tests Django isolent la base, mais pas forcément les variables globales ou les fichiers que votre code écrit sur disque. Évitez les effets de bord et utilisez des répertoires temporaires pour les tests de fichiers.

Pour observer les requêtes exécutées dans un test :

```
from django.test.utils import CaptureQueriesContext
from django.db import connection

with CaptureQueriesContext(connection) as queries:
    response = self.client.get(reverse("cours:liste"))
```

```
print(len(queries))
```

N'associez pas la qualité à un nombre arbitraire de requêtes : vérifiez que chaque requête est nécessaire et que le nombre ne croît pas avec le nombre d'objets.

Qualité automatique avec *ruff*

Ajoutez Ruff en dépendance de développement :

```
uv add --dev ruff
uv run ruff check .
uv run ruff format --check .
```

Vous pouvez corriger automatiquement le formatage :

```
uv run ruff format .
```

Un outil de qualité ne remplace pas les tests : il détecte surtout des problèmes de style, d'imports ou de constructions suspectes. Les tests ne remplacent pas non plus la revue de conception. Ils se complètent.

Une petite stratégie de test

Pour BiblioCours, commencez par les comportements risqués :

- les cours brouillons ne sont jamais visibles publiquement ;
- un slug inconnu renvoie 404 ;
- un visiteur ne peut pas créer ou supprimer ;
- les champs obligatoires sont refusés ;
- le formulaire valide redirige après sauvegarde ;
- la recherche ne provoque pas d'erreur avec une chaîne vide ;
- les migrations partent d'une base vide.

Les tests de modèle peuvent vérifier les propriétés locales. Les tests de vue vérifient la route, le statut, le template et les données affichées. Les tests d'intégration plus larges servent pour les parcours critiques. N'écrivez pas un test séparé pour chaque ligne de code ; écrivez-en pour les contrats qui doivent rester vrais.

Exercices

1. Ajoutez un test qui vérifie qu'un cours brouillon renvoie 404 sur sa page publique.
2. Ajoutez un test POST valide qui vérifie la redirection et l'objet créé.
3. Ajoutez un test POST invalide qui vérifie que la base ne contient aucun objet partiellement créé.
4. Mesurez le nombre de requêtes de la liste pour 1 puis 20 cours et corrigez une éventuelle croissance $N+1$.

Préparer la production et continuer l'apprentissage

Développement et production n'ont pas le même but

Le serveur `runserver` accélère l'apprentissage : il recharge le code, montre les tracebacks et sert certains fichiers statiques. Il n'est pas conçu pour recevoir du trafic public. En production, l'application est exécutée derrière un serveur applicatif et souvent un proxy qui gère TLS, compression, fichiers statiques et limites réseau.

Le déploiement n'est pas une commande magique. Il consiste à rendre reproductibles plusieurs éléments :

- la version de Python ;
- les dépendances ;
- les réglages et secrets ;
- le schéma de base ;
- les fichiers statiques ;
- le processus qui sert Django ;
- les sauvegardes et la supervision.

Ce chapitre prépare une mise en production et enseigne les contrôles communs à toute plateforme. Il ne décrit pas le déploiement complet chez un hébergeur particulier : les étapes de création du service, des variables secrètes, de la base de données et du domaine dépendent du fournisseur choisi. Avant une mise en ligne réelle, suivez aussi son guide officiel et testez la procédure sur un environnement de préproduction.

Le guide de déploiement Django rappelle que la configuration par défaut est adaptée au développement et doit être audité avant exposition publique ([Django Software Foundation 2025d](#)). Une démarche de déploiement progressive et documentée est également au cœur des ouvrages consacrés à Django en production ([Hajian 2022](#)).

Séparer la configuration du code

Une base minimale de réglages utilise l'environnement :

```
# config/settings.py
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.environ.get("DJANGO_DEBUG", "false").lower() == "true"

ALLOWED_HOSTS = [
    value.strip()
    for value in os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",")
    if value.strip()
]
```

En développement, vous pouvez fournir des valeurs temporaires dans le shell :

```
export DJANGO_SECRET_KEY="cle-locale-uniquement"
export DJANGO_DEBUG="true"
export DJANGO_ALLOWED_HOSTS="127.0.0.1,localhost"
uv run python manage.py runserver
```

Ne copiez pas ces valeurs dans `settings.py` si le dépôt peut devenir public. Le système de secrets de l'hébergeur est le bon emplacement en production.

Les réglages de sécurité courants comprennent :

```
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31_536_000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_CONTENT_TYPE_NOSNIFF = True
```

N'activez HSTS qu'après avoir vérifié que tout le domaine fonctionne en HTTPS : une mauvaise configuration peut rendre un domaine difficile à récupérer dans un navigateur. Les options doivent être adaptées à votre proxy et à votre hébergeur.

Exécuter les vérifications Django

Avant chaque déploiement :

```
uv run python manage.py check --deploy
uv run python manage.py test
uv run ruff check .
uv run python manage.py makemigrations --check --dry-run
```

La dernière commande échoue si des modifications de modèles ne sont pas accompagnées d'une migration. Elle évite de découvrir au moment du déploiement que le schéma de production est en retard.

Base de données et migrations

SQLite est excellente pour un premier projet et un apprentissage local. Une application multi-utilisateur en production utilisera souvent PostgreSQL ou une base gérée par l'hébergeur. Le modèle Django reste similaire, mais les opérations, les performances, les sauvegardes et les droits diffèrent.

Le processus de migration est :

```
uv run python manage.py migrate
```

Il doit être exécuté comme étape contrôlée du déploiement. Ne supprimez pas la base de production pour « repartir proprement » : une migration est une évolution du schéma, et la conservation des données fait partie du problème.

Une sauvegarde n'est utile que si sa restauration a été testée. Documentez :

- qui peut lancer une restauration ;
- quelle est la fréquence des sauvegardes ;
- quelle perte de données maximale est acceptable ;
- comment vérifier que l'application redémarre sur une copie restaurée.

Fichiers statiques

En production :

```
uv run python manage.py collectstatic --noinput
```

Le serveur applicatif ne devrait pas être responsable de servir efficacement chaque fichier CSS. Configurez un serveur web, un CDN ou un stockage objet pour `STATIC_ROOT`. Vérifiez ensuite qu'une page réelle charge le CSS avec le bon `STATIC_URL`.

Les médias envoyés par des utilisateurs nécessitent une stratégie séparée : stockage persistant, contrôle d'accès, limites de taille, politique de rétention et sauvegarde. Un disque local éphémère peut faire disparaître les fichiers à chaque redéploiement.

WSGI, ASGI et processus

Django peut être servi par WSGI pour les applications synchrones classiques :

```
uv add gunicorn
uv run gunicorn config.wsgi:application
```

ASGI permet aussi d'intégrer des usages asynchrones :

```
uv add uvicorn
uv run uvicorn config.asgi:application
```

Le choix du serveur et ses options dépendent de l'hébergeur et de la charge. Une fonction `async def` ne rend pas automatiquement toutes les opérations rapides : un appel bloquant à une base ou à une API peut toujours bloquer le worker. L'asynchronisme est une décision d'architecture, pas un mot-clé à ajouter partout.

Journalisation et erreurs

En production, désactivez les pages de debug mais conservez des journaux exploitables :

```
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {
        "console": {"class": "logging.StreamHandler"},
    },
    "loggers": {
        "django": {
            "handlers": ["console"],
            "level": "INFO",
        },
    },
}
```

Ne journalisez pas les mots de passe, tokens, cookies ou données personnelles inutiles. Un log est une copie de données qui peut vivre longtemps et être accessible à davantage de personnes que la base applicative.

Déployer avec un `pyproject.toml`

Le fichier doit déclarer les dépendances d'exécution. Dans un pipeline de déploiement, on peut utiliser :

```
uv sync --frozen
uv run python manage.py migrate
uv run python manage.py collectstatic --noinput
uv run python manage.py check --deploy
```

`--frozen` demande à `uv` de respecter `uv.lock` sans recalculer la résolution. Cela rend l'installation prévisible. Si `pyproject.toml` et `uv.lock` ne sont plus cohérents, le pipeline doit échouer : il ne faut pas réparer silencieusement la production en choisissant de nouvelles versions.

Une checklist finale

Avant de considérer BiblioCours comme publiable :

```
[ ] DEBUG est désactivé
[ ] SECRET_KEY et mots de passe viennent d'un gestionnaire de secrets
[ ] ALLOWED_HOSTS et CSRF_TRUSTED_ORIGINS sont corrects
[ ] HTTPS est actif
[ ] les migrations sont versionnées et appliquées
[ ] les fichiers statiques sont collectés et servis
[ ] les médias sont persistants et protégés
[ ] les tests passent dans un environnement propre
[ ] les logs ne contiennent pas de secrets
[ ] la base et les médias sont sauvegardés
[ ] une procédure de restauration est documentée
[ ] le serveur de développement n'est pas le serveur public
```

Cette liste ne remplace pas un audit. Elle fournit un point de départ concret pour ne pas oublier les réglages qui restent invisibles pendant le développement local.

Aller plus loin : une progression raisonnable

Une fois le catalogue maîtrisé, ajoutez une seule capacité à la fois :

1. **vues génériques et class-based views** pour réduire le code répétitif;
2. **comptes et profils** avec un modèle utilisateur adapté avant la première migration d'un nouveau projet ;
3. **recherche et filtres** avec pagination et indexation mesurées ;
4. **API JSON** avec Django REST Framework lorsque d'autres clients doivent consommer les données ;
5. **tâches asynchrones** pour les courriels et traitements longs ;
6. **observabilité** pour connaître les temps de réponse et les erreurs ;
7. **CI** pour exécuter tests, vérifications et migrations sur chaque modification.

Il est préférable de comprendre une petite application de bout en bout que de superposer très tôt dix bibliothèques dont on ne connaît pas les responsabilités. La documentation de Django, du langage Python et des outils utilisés doit accompagner cette progression ([Python Software Foundation 2025](#) ; [Encode 2025](#)).

Projet de synthèse

Terminez BiblioCours avec les exigences suivantes :

- une personne anonyme peut consulter les cours publiés ;
- un cours brouillon n'est jamais affiché sur une URL publique ;
- un éditeur authentifié peut créer et modifier un cours ;
- une matière ne peut pas être supprimée si elle contient des cours ;
- le formulaire affiche ses erreurs sans perdre les données valides ;
- les pages utilisent des URLs nommées et des templates hérités ;
- les tests couvrent le catalogue, les permissions et les formulaires ;
- `uv sync --frozen` permet de recréer l'environnement ;
- `check --deploy` n'affiche plus de problème connu non traité.

À la fin du parcours, les fichiers principaux doivent se répartir ainsi (certains fichiers générés par Django ne sont pas représentés) :

```
bibliocours/  
  .env.example  
  pyproject.toml  
  uv.lock  
  manage.py  
  config/  
    settings.py  
    urls.py  
  cours/  
    migrations/  
    static/cours/site.css  
    templates/cours/  
    tests/  
    admin.py  
    forms.py  
    models.py  
    urls.py  
    views.py  
  templates/registration/login.html
```

Le fichier `.env` local, `.venv/`, `db.sqlite3` et `staticfiles/` ne sont pas versionnés. Comparez cette carte à votre dépôt ; si un fichier est ailleurs, vérifiez que le réglage ou le chemin des templates et des fichiers statiques correspond.

Pour chaque fonctionnalité, faites un commit qui raconte une décision :

```
git add .  
git commit -m "feat: protéger la création des cours"
```

Un historique lisible devient une documentation supplémentaire du projet. Il permet de comprendre non seulement l'état actuel, mais aussi la raison des évolutions.

Exercices

1. Configurez un environnement de test séparé et faites tourner la suite depuis une base vide.
2. Simulez une variable manquante comme `DJANGO_SECRET_KEY` et rendez l'erreur compréhensible.
3. Écrivez une procédure de restauration locale à partir d'une sauvegarde SQLite de test.
4. Documentez dans le README les commandes `uv sync --frozen`, `migrate`, `collectstatic`, `check --deploy` et `test`.

References

- Astral. 2025. « Uv Documentation ». <https://docs.astral.sh/uv/>.
- Django Software Foundation. 2025a. « Cross Site Request Forgery Protection ». <https://docs.djangoproject.com/en/5.2/howto/csrf/>.
- Django Software Foundation. 2025b. « Django Documentation ». <https://docs.djangoproject.com/en/5.2/>.
- Django Software Foundation. 2025c. « Django Overview : Design Philosophies ». <https://docs.djangoproject.com/en/5.2/misc/design-philosophies/>.
- Django Software Foundation. 2025d. « How to Deploy Django ». <https://docs.djangoproject.com/en/5.2/howto/deployment/>.
- Django Software Foundation. 2025e. « Models ». <https://docs.djangoproject.com/en/5.2/topics/db/models/>.
- Django Software Foundation. 2025f. « Security in Django ». <https://docs.djangoproject.com/en/5.2/topics/security/>.
- Django Software Foundation. 2025g. « Testing in Django ». <https://docs.djangoproject.com/en/5.2/topics/testing/>.
- Django Software Foundation. 2025h. « The Django Template Language ». <https://docs.djangoproject.com/en/5.2/ref/templates/language/>.
- Django Software Foundation. 2025i. « Working with Forms ». <https://docs.djangoproject.com/en/5.2/topics/forms/>.
- Django Software Foundation. 2025j. « Writing Your First Django App ». <https://docs.djangoproject.com/en/5.2/intro/tutorial01/>.
- Encode. 2025. « Django REST Framework ». <https://www.django-rest-framework.org/>.
- Greenfeld, Daniel Roy, et Audrey Roy Greenfeld. 2017. *Two Scoops of Django : Best Practices for Django 1.11*. 1 éd. Two Scoops Press.
- Hajian, William S. Vincent. 2022. *Django for Professionals : Production Websites with Python and Django*. 4 éd. Independently published.
- OWASP Foundation. 2021. « OWASP Top 10 : 2021 ». <https://owasp.org/Top10/>.
- Percival, Harry, et Bob Gregory. 2024. *Test-Driven Development with Python*. 3 éd. O'Reilly Media.
- Python Software Foundation. 2025. « Python 3 Documentation ». <https://docs.python.org/3/>.